Information and Software Technology 93 (2018) 223–245

Contents lists available at ScienceDirect

Information and Software Technology

journal homepage: www.elsevier.com/locate/infsof

A systematic literature review of software requirements reuse approaches

Mohsin Irshad*, Kai Petersen, Simon Poulding

Blekinge Institute of Technology, Karlskrona, Sweden

ARTICLE INFO

Keywords:
Software requirements
Requirements reuse
Rigor
Relevance
Artefact reuse
Reusability

ABSTRACT

Context: Early software reuse is considered as the most beneficial form of software reuse. Hence, previous research has focused on supporting the reuse of software requirements.
Objective: This study aims to identify and investigate the current state of the art with respect to (a) what requirement reuse approaches have been proposed, (b) the methods used to evaluate the approaches, (c) the characteristics of the approaches, and (d) the quality of empirical studies on requirements reuse with respect to rigor and relevance.
Method: We conducted a systematic review and a combination of snowball sampling and database search have been used to identify the studies. The rigor and relevance scoring rubric has been used to assess the quality of the empirical studies. Multiple researchers have been involved in each step to increase the reliability of the study.
Results: Sixty-nine studies were identified that describe requirements reuse approaches. The majority of the approaches used structuring and matching of requirements as a method to support requirements reuse and text-based artefacts were commonly used as an input to these approaches. Further evaluation of the studies revealed that the majority of the approaches are not validated in the industry. The subset of empirical studies (22 in total) was analyzed for rigor and relevance and two studies achieved the maximum score for rigor and relevance based on the rubric. It was found that mostly text-based requirements reuse approaches were validated in the industry.
Conclusion: From the review, it was found that a number of approaches already exist in literature, but many approaches are not validated in industry. The evaluation of rigor and relevance of empirical studies show that these do not contain details of context, validity threats, and the industrial settings, thus highlighting the need for the industrial evaluation of the approaches.

1. Introduction

Software reuse describes the practice of reusing existing software artefacts for developing a new product or maintaining an old product [70]. The field has been the subject of research for several years and different aspects of software reuse such as costs, artefacts, the level of reuse, stakeholders, and reuse processes have been investigated [71–74,97]. Software reuse can take place at any time during a project's lifecycle starting from the requirements analysis phase to the maintenance phase and researchers have pointed out that the reuse in early phases of the project leads to more benefits [73,75]. The reuse of software requirements is one of the approaches to enable early reuse. Software requirements contain the information on the needs of a user or the details of a contract that is formally imposed. The reuse of software requirements has been the focus of investigations in the past and the reuse of requirements can help in two ways: (i) reducing the time for the analysis of requirements; (ii) identifying reusable code and test artefacts having similar requirements, thus resulting in an early reuse i.e., early in development cycle [76,77].

Various studies on reusing software requirements are present in the literature [4,42,55]. Approaches to requirements reuse take different forms of requirements into consideration, such as text-based requirements, use-cases, and formal specifications. Different strategies such as variable/ configurable part of requirements or analogy based matching have been proposed to achieve requirements reuse [19,78]. Similarly, formal methods based requirements reuse approaches have been proposed [101]. These approaches are applied in various different contexts and use different reusable artifacts to increase the reusability. This study is an attempt to identify and analyze the requirements reuse approaches present in the literature.

Empirical investigations in software engineering allow practitioners and researchers to evaluate a solution before it is applied in real world settings. This can help in assessing the benefits and limitations of a solution. Each solution (a model, guideline or method) should be described with details that others can use to reproduce the solution. Kitchenham et al. point to the need of improving the reporting of

* Corresponding author.
E-mail addresses: mohsin.irshad@bth.se (M. Irshad), kai.petersen@bth.se (K. Petersen), simon.poulding@bth.se (S. Poulding).

http://dx.doi.org/10.1016/j.infsof.2017.09.009
Received 21 November 2016; Received in revised form 11 September 2017; Accepted 22 September 2017
Available online 23 September 2017
0950-5849/ © 2017 Elsevier B.V. All rights reserved.

empirical research in software engineering and concluded that guidelines for reporting empirical research are required [87]. Later, they have described guidelines for reporting context, design, data collection, analysis, presentation and interpretation of results of an empirical study [87]. Mohagheghi et al. carried out a literature review of empirical studies on software reuse and found that several studies lack information on research design, hypothesis, and research questions [74]. They found that many empirical studies lack vital information for the readers and comprehensive discussion on the results gathered during the study. Jedlitschka et al. describe the scarce empirical studies and lack of guidelines for reporting an empirical study [88]. Later they propose a method for the aggregated reporting of empirical studies.

The empirical studies on requirements reuse approaches are reported for the industry and research community. There is a need to evaluate the quality of these studies, as is done for other software engineering fields [74,87,88]. Ivarsson and Gorschek proposed a model to evaluate the empirical studies [79]. The model helps in quantifying important aspects of a study such as a hypothesis, research methods, subjects, context etc. Rigor describes the way a study is carried out and reported. The relevance describes the potential impact of the study on academia and industry. In this case, Ivarsson and Gorschek have referred to the industrial relevance and the studies can be evaluated based on their evaluation rubric [79]. The results of the empirical evidence are relevant for the practitioners if they have been obtained in a realistic context with regard to environment, subjects, and tasks [80]. Evidence based software engineering (EBSE) attempts to help the practitioners in understanding and interpreting the results of scientific studies i.e., finding the best questions and dissecting the available evidence [81]. Systematic literature reviews (SLRs) are an important part of EBSE that support the identification and synthesis of evidence. In this study, we have utilised the systematic literature reviews for systematically identifying and synthesizing the relevant studies. We have excluded the studies that are on the reuse of knowledge related to requirements (e.g., capturing of past experiences and best practices to manage requirements) and included the studies on reuse of requirement artefacts.

The motivation of this study was to identify and evaluate the current state of the art for software requirements reuse approaches. We conducted an SLR on requirement reuse approaches and performed quality evaluations of the empirical studies based on the rigor and relevance rubric [79]. The target audience of this study is practitioners and researchers who want to evaluate the advantages and disadvantages of requirements reuse approaches. The main contributions of this study are:

- **Contribution 1:** To identify requirements reuse approaches presented in the literature. Approaches may refer to a model, method or guidelines for reusing software requirements.
- **Contribution 2:** Determine how requirements reuse approaches have been evaluated (experiments, case studies etc.,) and elicit the outcomes of the evaluations.
- **Contribution 3:** To evaluate the quality of the identified empirical studies based on the rigor and relevance rubric. Each empirical study (case study or experiment) is evaluated based on the rigor and relevance criteria proposed by Ivarsson and Gorschek [79].

The remainder of the paper is structured as follows. In Section 2 we describe the background and related work. The research methods used in the study are described in Section 3 and results are described in Section 4. Later, we discuss and analyze the findings of the study in Section 5. The study concludes by describing the conclusions in Section 6.

2. Related work

This section narrates the background and related work on requirements reuse approaches and quality evaluation in software engineering. We have examined previous primary, secondary and tertiary studies. A primary study describes a new, original idea and a secondary study summarizes existing primary studies. A tertiary literature review is a systematic study (e.g., a literature review) of secondary studies [98]. Two tertiary studies on requirements engineering were examined to identify any systematic literature reviews (SLRs) on reuse of requirements[99,100]. This section analyzes two previous systematic literature reviews on reuse of software requirements [76,96] and discusses different methods for evaluating the quality of studies.

2.1. Secondary studies on requirements reuse approaches

Several approaches exist in the literature that propose methods and guidelines for reusing requirements. These requirements reuse approaches have tried to address the reuse problems in different ways such as analogy-based reuse, viewpoints based reuse, use-cases reuse etc. These approaches take into consideration different requirement artefacts to process information related to software requirements.

Two tertiary studies on requirements engineering were inspected to identify any previous systematic literature reviews on requirements reuse. A tertiary study (published in 2014) by Bano et al. listed the previous systematic literature reviews (SLRs) on requirements engineering topics [99]. This study was reviewed (scanning the reference list, results and citations in Google Scholar) to identify the existing systematic literature reviews on reuse of software requirements. The tertiary study identified one previous SLR on reuse of requirements [96] and that literature review is analyzed in Table 1. Another study by Babar et al., on SLRs conducted in requirements engineering (also published in 2014), was reviewed (scanning the reference list, results, and citations in Google Scholar) to identify the SLRs on the reuse of software requirements [100]. This study did not discuss any existing literature reviews on reuse of requirements. Furthermore, during the execution of this study, we have tried to locate existing systematic literature reviews on reuse of software requirements found during the forward snowball sampling.

Limited work has been done on identifying the requirements reuse approaches present in the literature. We have found two previous studies that have attempted to address this issue [76,96], in a systematic way. Many studies in literature discuss requirements reuse approaches but the identification and analysis of previous studies on requirements reuse were not described in these studies [2,30,83]. In this section, we have analyzed the two existing systematic literature reviews (SLR). These two studies were identified during the execution of snowball sampling [76] and during the review of tertiary studies [96].

Azambuja et al. conducted a review of the state of art in requirements reuse approaches and identified 18 primary studies [96]. The study only includes the articles published between 2004 and 2009. Azambuja et al. categorize these identified studies based on the research methodologies used, the solution these studies describe and the application of these studies. The study concludes that there are various diverse requirements reuse approaches present in literature and this diversity is a challenge for adopting standard requirements reuse process.

Mussbacher and Kienzle reported a systematic literature review to identify articles on requirements reuse published in the Requirements Engineering Conference (RE) [76]. According to the authors, this literature review was limited to requirements reuse studies from RE Conferences [76]. The review identified sixty-nine studies that fit the inclusion criteria. That is, the considered publications were restricted to the publication years of 1993–2013 and excluded text-based requirements. According to study, there are seven types of requirements reuse methods: impact, matching, patterns, selection, SPL, traceability, and transformation. The study further describes the research agenda based on the results of a literature review.

A summary of the previous literature reviews is presented in

Table 1
Summary of the previous studies.

Study	objective	Primary / Secondary study	Systematically conducted	Number of identifiedstudies	Quality evaluation conducted	Inclusion / Exclusion criteria provided explicitly	Outcome for requirements reuse
[76]	To investigate that requirements reuse requires a coordinated effort spanning the complete software development life cycle.	Secondary	Yes	69	No	No	Classifies the reuse approaches on the basis of requirements artefacts and the approaches proposed. Only covers studies from RE conferences.
[96]	Review the state of art in requirements reuse approaches.	Secondary	Yes	18	No	No	List of reuse approaches and the types of solution described by the studies. Only includes studies between year 2004–2009.

Table 1. The column 'Primary / Secondary Study' describes if a study describes new original idea (Primary study) or the study summarizes existing primary studies (Secondary study). The column 'Systematically Conducted' describes if the study identifies existing literature using the systematic literature review guidelines. The column 'Quality Evaluation Conducted' describes if the quality of the results and claims of primary study are critically analyzed using a defined criteria.

The summary of previous studies (in Table 1) shows that previous literature reviews discussing different requirements reuse approaches: (i) do not explicitly describe the inclusion/exclusion criteria despite conducting the study in a systematic way; (ii) the studies do not conduct the quality evaluation of the identified primary studies; (iii) one of the secondary studies [96] only includes the studies between 2004 and 2009; (iv) one of the secondary studies [76] only includes the studies published in the Requirements Engineering Conference and (v) the number of identified primary studies vary among the previously conducted studies. In order to address theses, a systematic literature review is performed in this study and quality evaluation of empirical studies is conducted.

2.2. Quality evaluation through rigor and relevance scoring in secondary studies

The research community has proposed different methods for evaluating the quality (or impact) of a study, e.g., counting the citations, rigor, and relevance [79,82,90]. However, there is no agreed-upon method to evaluate a research study. In this study, we have used the method proposed by Ivarsson and Gorschek [79] to evaluate empirical research studies. The strength of the quality assessment method proposed by Ivarsson and Gorschek [79] is that it takes two dimensions into consideration, namely rigor and relevance. Rigor refers to *"to both how an evaluation is performed and how it is reported"* [79]. Relevance refers to *"the potential impact the research has on both academia and industry"* [79]. Sjøberg et al. highlight realistic environments, subjects, and tasks [80] as important to achieving relevant research results, which are reflected in the assessment method by Ivarsson and Gorschek. Multiple studies have utilised Ivarsson and Gorschek's rubric and also provided refinements (e.g., [89,91–93]); the rubric has proven to be useful to distinguish and categorize studies based on their rigor and relevance in these studies.

3. Research methodology

The research methodology of this study is based on various guidelines present in the literature. We used the guidelines proposed by Kitchenham and Charters for conducting systematic literature reviews [84] complemented by the recent guidelines on snowball sampling by Wohlin [85] and quality assessment guidelines proposed by Ivarsson and Gorschek [79].

In the following sub-sections, we have described the execution of the steps proposed by Kitchenham and Charters to conduct the systematic literature review [84]. The steps are: (1) define the review questions, (2) define the need for the review, (3) identify the primary studies, (4) assess the quality of the studies, (5) perform the data extraction, and (6) identify the validity threats.

3.1. Review questions

This work is based on three research questions that are linked to the contributions of this study.

RQ 1: What are the different software requirements reuse approaches present in literature? *The answer to this question will provide us a list of requirements reuse approaches and their characteristics. That is, it serves as a corpus of alternative methods to achieve requirements reuse. (Contribution 1)*

RQ 2: How have the requirements reuse approaches identified in RQ1 been evaluated, and what was the outcome? *The answer to this question shows which research methods have been used for the evaluation. We also identify what the outcomes of the evaluation were. (Contribution 2)*

RQ 3: What was the rigor and relevance of empirical studies on requirement reuse approaches? *The answer to this question identifies the strengths and weaknesses of empirical studies. (Contribution 3)*

3.2. Need for the review

From the study of the related work we conclude that: (i) the secondary studies describing the previous work on requirements reuse approaches have not reported the methodology used for carrying out the study; (ii) the studies that discuss the requirements reuse approaches have not assessed the quality of the research on requirements reuse approaches; (iii) the requirements reuse approaches are seldom evaluated in an industry setting as reported by the studies [76,94]. To address (i) and (ii) we conducted a systematic review and utilised the evaluation rubric by Ivarsson and Gorschek to assess the quality of primary studies [79]. To address (iii) we highlighted those studies evaluated in an industry and provide an overview of the evaluation results for each study.

3.3. Identification of primary studies

The identification of primary studies was conducted as follows:

1) Developed a start set using keywords associated with research questions.
   a) Select candidates for the start set.
   b) Revise the candidates based on the review and discussion between the authors.
   c) Final start set is selected.
2) Developed inclusion and exclusion criteria.
3) Author one performed backward and forward snowball sampling on each study in the start set, based on the inclusion criteria.
   a) Screened all the titles of studies in the snowball sampling and tracked these with the help of a spreadsheet (Excel).
   b) If the title of any study was not conclusive, then the abstract was read before accepting or rejecting the study.
   c) If the abstract was not conclusive then the conclusion was read before accepting or rejecting the study.
   d) In few cases where the conclusion was ambiguous then the complete study was read before accepting or rejecting the study.
4) Author one performed the same procedure during each iteration until the final list of studies was obtained i.e., no new studies were found.
5) The second author performed a sanity check on the randomly selected studies from a set containing accepted and rejected studies.
6) The second author performed a database search to increase the confidence in the snowball sampling procedure, described in Section 3.3.4.

We aimed to find the requirements reuse approaches described in the literature. Database search and snowball sampling are two well-known methods for identification of literature studies [84,85]. Badampudi et al. claimed that snowball sampling was more effective during the identification of research studies [86]. Therefore, we decided to use snowball sampling as the primary method to identify the studies. Database search was also used (for a sanity check) to increase the confidence in the results from the snowball sampling. The guidelines for conducting the snowball sampling are proposed by Wohlin [85] and have been used in several studies [109,110].

3.3.1. Develop a start set

The first phase is to identify a start set for executing the snowball sampling. According to Wohlin, there is no agreed upon method to select the start set but he has provided recommendations for the identification of the start set [85]. The following criteria were used for developing the start set:

- The selected articles are from different authors.
- The selected articles are from different publication years, covering as much time span as possible.
- The selected articles describe a study on the reuse of software requirements.
- At least one selected article is on the reuse of use-cases.
- At least one selected article contains a previous attempt at reviewing the requirements reuse studies.

As per the guidelines to conduct snowball sampling, Google Scholar is used for identifying and locating the articles to satisfy the above criteria for the start set to reduce bias [85]. The keywords “Requirements”, ‘Use cases” and “reuse” were used to identify the initial list of articles from Google Scholar (Search Term 1: Requirements AND Reuse, Search Term 2: Use cases AND Reuse) before the application of Wohlin’s recommendations. “Search Term 1” and “Search Term 2” were searched separately and a preliminary set of thirty articles was selected based on both search terms. Only the first ten pages (each page listing twenty results) of search results from Google Scholar were examined for relevant studies during March 2016. This preliminary list was selected based on the criteria defined above (based on the guidelines proposed by Wohlin [85]) and after reading the articles. The first author performed these steps and a previous systematic literature review was

Table 2
Start set for snowball sampling.

No	Articles in start set	Search term used in Google Scholar
i	A practical approach to requirements reuse in product families of on-board systems. [S4] [4]	Requirements and reuse. No filter.
ii	A systems approach to product line requirements reuse. [S8] [8]	Requirements and reuse. No filter.
iii	An integrative Approach to Requirements Analysis: How task models support Requirements reuse in user-centric design framework [S13] [13]	Requirements and reuse. No filter.
iv	Analogical reuse of requirements frameworks. [S14] [14]	Requirements and reuse. No filter.
v	Evaluation of a Systematic Approach to Requirements Reuse. [S17] [17]	Requirements and reuse. No filter.
vi	Reuse of Requirements reduced time to market at one Industrial Shop: A case Study. [S41] [41]	Requirements and reuse. No filter.
vii	Sharing And Reuse Of Requirements Knowledge. [S49] [49]	Requirements and reuse. Filter “before 1992”.
viii	Supporting reusable use cases. [S56] [56]	Use cases and reuse. No filter.
ix	Ten Steps Towards Systematic Requirements Reuse. [94]	Requirements and reuse. No filter.
x	Identified Adjustability Dimensions when Generating A Product Specific Requirements Specification by Requirements Reuse [95]	Requirements and reuse. Filter “since 2014”.
xi	Systematic Review of Requirements Reuse [96]	Not searched. Authors were aware of this SLR from [99].

added to this preliminary list. Later, all the authors reviewed the preliminary list of thirty studies (selected based on criteria) and a mutual agreement was reached on the final start set of eleven studies.

The start set of eleven studies is listed in Table 2 along with the search term used for the identification. These eleven studies were used to initiate the snowball sampling. The first eight studies, in Table 2, of the start set were part of the final list of primary studies, listed in Section 7. The three articles in the start set [94–96] were not included in the final list of primary studies, listed in Section 7, as these were not describing a requirement reuse approach (e.g., existing SLR or a study that does not propose any requirements reuse approach). These three articles [94–96] were included as part of the start set since these facilitated the identification of other relevant studies during snowball sampling.

3.3.2. Develop inclusion and exclusion criteria

The next phase was to develop inclusion and exclusion criteria to be applied on the articles found during forward and backward snowball sampling. The criteria are provided below:

Inclusion criteria: A study is included based on the following criteria:

- Study describes a software requirement reuse approach.
- Study is published in a peer-reviewed journal, conference or workshop.

Exclusion criteria: Studies are excluded based on the following criteria:

- Do not belong to software engineering/computer science.
- Do not discuss software reuse approaches in requirements.
- Discusses reuse of requirements knowledge but does not describe an approach for the reuse of software requirements.
- Discusses requirements elicitation approach but does not describe the reuse aspects in detail.
- Full text of the study was not available.
- Described in a language other than English.
- Study is not peer-reviewed.

3.3.3. Execution of the snowball sampling procedure

To the start set, we applied the inclusion and exclusion criteria using forward (identifying citing articles) and backward snowball sampling (identifying cited articles). Google Scholar was used for identifying the citing articles. In total five iterations were performed to the point where no new articles were found. Each article in the start set was first read in order to understand it. Later, the reference list was searched manually for references. The first author read the title of the article and if it was deemed to be relevant the article was included. In cases where the article's title was not clear, the article was downloaded and its abstract was read before making a decision. Duplicate articles were removed at the time of recording the relevant articles in Excel sheet. This detection and removal of the articles were done manually. An article was only noted once in the Excel sheet and if it appeared in other reference lists it was ignored.

It should be noted that a sanity check was performed on the execution of the snowball sampling. An author (not part of snowball sampling) executed inclusion and exclusion criteria on a sample of 140 selected studies (containing seventy articles selected during snowball sampling and seventy randomly selected articles identified during snowball sampling but not meeting inclusion criteria defined in Section 3.3.2). There was an agreement on the majority of the articles that were included as a primary study or excluded as not relevant (see Table 3). The remaining articles were discussed among authors and a mutual agreement was reached on disparities. No new article was added, however one article [76] was excluded from the list of primary studies and classified as secondary study. The sanity check was found to be satisfactory and execution was checked against the guidelines proposed by Wohlin [85]. Table 3 contains the results of this sanity check where "Author 1" represents the author that carried out snowball sampling and "Author 2" represents the reviewer that performed the sanity check.

Table 3
Snowball sampling sanity check.

	Author 1 Includes	Author 1 Excludes	Author 1 Uncertain
Author 2 Includes	43	7	1
Author 2 Excludes	1	87	0
Author 2 Uncertain	0	1	0

As mentioned earlier Microsoft Excel was used for keeping track of the snowball sampling procedure. That is, the title, authors, and the publication forum were extracted for each article selected. Every new iteration was noted within a new Excel worksheet. The sheets were named "iteration 1", "iteration 2" etc. In this way, a traceability of the articles was possible. When the execution of the study was complete all the articles were copied into a single Excel sheet.

We identified 69 studies through the snowball sampling method. Five iterations were required before reaching saturation. The numbers of studies found during each of the iteration through forward snowball as well as backward snowball sampling are shown in Fig. 1. The final list of primary studies is presented in Section 7.

3.3.4. Validation through database search

An additional sanity check was performed by searching in the database (Scopus) with a combination of search strings "Requirements" and "Reuse", excluding subject areas other than computer science and software engineering. The author who performed the sanity check on snowball sampling performed this step. A total of 89 studies were found in the search result. The inclusion and exclusion criteria were applied to these studies and fourteen (14) primary studies were identified. However, no new study was found that was not one of the studies found by snowball sampling. It is interesting to note that the search conducted found many of the articles similar to snowball sampling, but not all the articles found by snowball sampling. Badampudi et al. have described similar findings in their study when snowball sampling outperformed database search during the identification of relevant studies [86]. This sanity check increased the confidence in the snowball sampling and identified primary studies. There is a possibility that searching in a different database (e.g., Google Scholar) may have revealed new articles not appearing in our snowballing results as compared to Scopus.

3.4. Evaluation of study quality

The evaluation of empirical studies was performed in this phase. These empirical studies were extracted from the final list of sixty-nine primary studies that were identified during snowball sampling. The two subsets of empirical studies (15 case studies listed in Table 17 and 7 experiments listed in Table 18) were assessed using the rigor and relevance rubric by Ivarsson and Gorschek [79].

The first step was to decide the criteria for evaluation of rigor and relevance. The first author developed the initial draft of the criteria based on the proposal by Ivarsson and Gorschek [79]. The definitions of the criteria also took into consideration the refinements proposed by Munir et al. (such as types of Research Methods etc.,) [91]. Later, the other authors evaluated the criteria and amendments were proposed to the criteria. These amendments were mainly concerned with the evaluation of context for the rigor and scale for the relevance.

3.4.1. Criteria for rigor evaluation

The criteria for rigor evaluate the reporting of study context, study

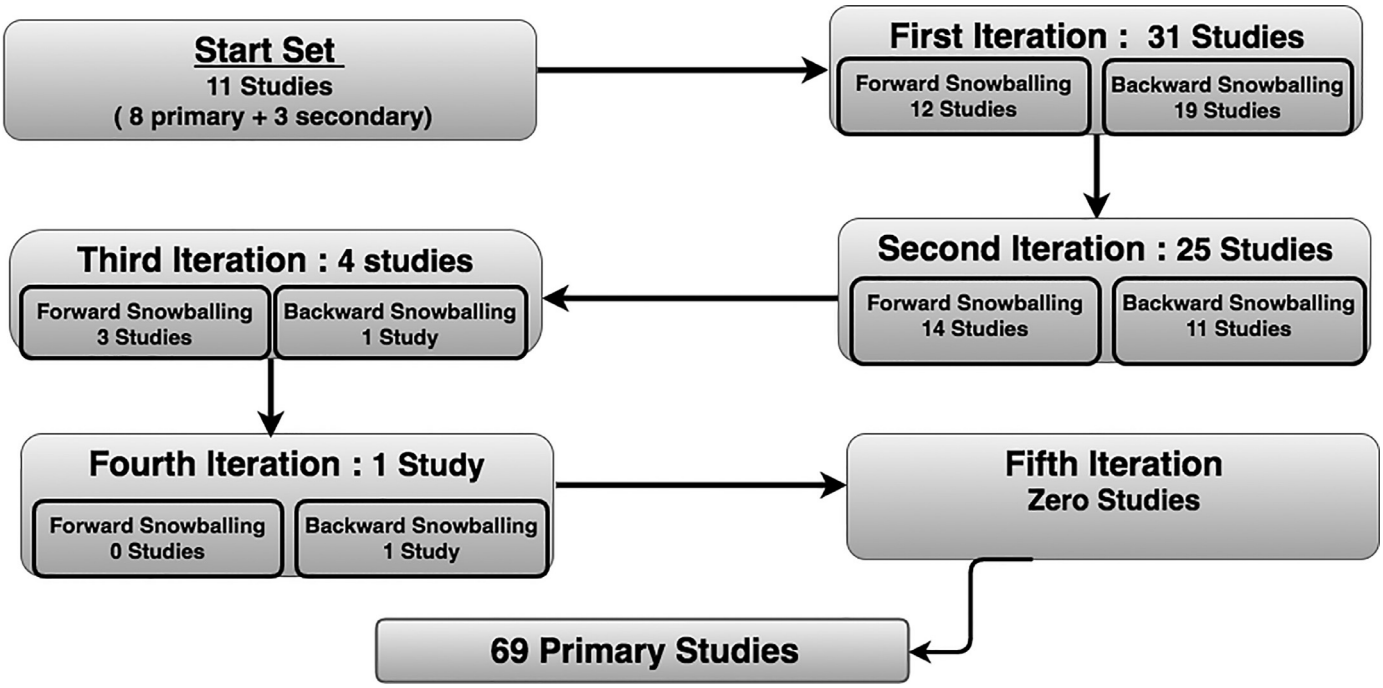

Fig. 1. Results of snowball sampling iterations.

Table 4
Rigor evaluation criteria.

Rigor rubrics, inspiration from [79]			
Rigor	Context	Study design	Validity
Strong = **1**	Motivation or hypothesis, background of the problem, related studies, subjects, settings (industry, laboratory), organisation details, tool support if any, constraints	Study describes research method, data collection, discussion on the results, motivation for the research method, details of the measurements used in the study.	If all four types (internal, external, conclusion and construct validity) of validity threats are discussed.
Moderate = **0.5**	If any of the above factors are missing.	If any of the above factors are missing.	If some description of validity threats is present
Weak = **0**	If none of the above factors are discussed.	No description of studies' research design is present.	If validity threats are not discussed at all.

design, and validity threats. The criteria and corresponding evaluation points are provided in Table 4.

- *Context:* The context describes the background and the settings in which a study took place. The context of study helps the reader in understanding the environment and motivation behind the study so that results are interpreted in that context.
- *Study design:* The study design describes the research approach used in the study. This involves describing the details of the research method, the implementation of the research design, the variables measured during the study and the motivation behind the design.
- *Validity threats:* The threats to the validity of the studies help in understanding and interpreting the result and their applicability. Kitchenham et al. pointed out the importance of discussing the validity threats in empirical studies and implications of not describing the threats to the validity of the study [87].

3.4.2. Criteria for relevance

The criteria were developed in order to systematically evaluate the relevance to the industry for the studies. The criteria evaluate four factors: subjects of the study, the context of the study, the scale of the study and research method used in the study. The detailed criteria are present in Table 5.

- *Subjects:* Sjøberg et al. described the importance of having realistic settings when conducting industrial studies [80]. The proposed way of carrying out industrial studies was to use subjects from industry. Therefore, subjects are an important aspect of the relevance.
- *Context:* The industrial relevance of the study is higher if the study has taken place in industry. In the scope of this study, if the requirements reuse method was evaluated in the industry then it has a higher contribution towards relevance thus higher context value.
- *Scale:* The scale of the study describes the level of complexity and realistic settings used to evaluate the requirements reuse methods. The requirements reused were evaluated to see if these are a representation of the real system or example applications.
- *Research method:* The relevance of the study for the practitioners and industry requires that the research method should also be relevant to the industry [79].

3.4.3. Execution of criteria on identified studies

In order to facilitate the execution of the criteria on identified studies, two data extraction forms were developed for rigor and relevance evaluation. These forms were developed in Excel and were based on the criteria described in the previous section.

After developing the criteria and data extraction forms, the identified articles were read and evaluated. While reading an article the data was extracted and filled in the form. During the extraction, the important sections in each article were highlighted and annotated in order to keep track of the evaluated numbers. The first author performed this

Table 5
Relevance evaluation criteria.

Relevance rubrics, inspiration from [79]				
Relevance	Subject	Context	Scale	Research method
Strong = **1**	The subjects are from industry and they are the users of the requirements reuse approach.	The evaluation is performed in the industry.	The requirements, considered by approach, are from industry i.e., complexity and representation of real system.	The research method used are either of the following: Action Research, Lessons Learned, Case Study, Experiments, Field Study, Interview / Survey, Descriptive / Exploratory Study.
Moderate = **0.5**	The subjects are mix of industry and non-industry. They are the users of the requirement reuse approach.	The evaluation is performed partially in the industrial setting and partially in the non-industrial setting.	The requirements, considered by approach, are from an example application i.e., complexity and representation not of a real system.	Not Applicable
Weak = **0**	No details about the subjects are present or subjects are students.	The evaluation is not performed in the industry.	There were no details mentioned about the requirements and their complexity.	Research method used is not from the above mentioned research methods or the article is not clear about the research method used.

step.

First, the articles were evaluated for rigor by the first author. Each article was evaluated twice, during the evaluation of rigor, in order to ascertain the correct extraction and assignment of rubrics values, known as 'test-retest check' as suggested by Kitchenhamn [106]. The data related to validity threats was extracted in the start as a 'pilot extraction'. This was done in order to get familiar with data extraction process. Later, the data related to the context and study's design were extracted. After completing the evaluation, a second iteration was conducted by the same author (first author) in order to check *intra-rater* reliability (known as test-retest check [106]). No changes were required because of the second evaluation indicating *intra-rater* reliability. These two iterations helped in increasing the confidence in evaluation and assessment, done as a part of *intra-rater* reliability.

In the second phase, the relevance of each identified article was evaluated. The evaluation was conducted twice, initially for evaluation of criteria and later to confirm *intra-rater* reliability (test-retest check [106]). In the first evaluation, each article was one by one evaluated based on the relevance criteria and the numbers were assigned for each factor. Later, to re-check the evaluation the first author performed the second iteration. No numbers were changed in the second evaluation. The first author also performed this step.

The other authors (second and third) performed the evaluation's sanity check. This sanity check was conducted to increase the confidence in the evaluation, indicating *inter-rater* reliability. The author performing the initial evaluation was not part of this review (sanity check). The two authors of the study performed this review, independently. The differences (such as different evaluation numbers) were discussed and a mutual agreement was reached on the differences.

3.5. Data extraction and synthesis

Data extraction form, developed in Excel, was used to record the findings of each primary study. Data extraction consisted of three reading iterations. Initially, a study was read completely and important aspects (presented in Table 6) were highlighted for the second review. In the second reading, the highlighted aspects were re-confirmed and captured in the data extraction form (Excel sheet). The unclear or missing information was documented in the comments section. A third, and final, review was conducted to verify that the data extraction was correctly done and to address the issues found during the first two reading reviews.

The above data extraction step resulted in an Excel sheet containing the required information from all the primary studies. The data collection sheet is provided online [112]. Two new Excel sheets, containing only the empirically evaluated studies, were retrieved from the main sheet. The purpose of this step was to carry out the rigor and relevance evaluation as described in Section 3.4.

For the synthesis we utilised content analysis in order to structure the results, assigning studies and their results to categories [107]. Furthermore, narrative synthesis and summaries are presented for each research question.

Table 6
Extracted fields from the studies.

Approach	Details
General Information	Title, Year, Publication Type, Comments
Research Method	Experiment, Case Study, Example Application, Lessons Learned
Reuse Characteristics	Reuse Approach Type, Artefact Type, Tool Support (Yes / No), Repository (Yes / No), Stakeholders Involvement (Yes / No), Product Lines Support (Yes / No)

3.6. Threats to validity

This section contains a discussion on the validity threats based on the categorization provided by Petersen and Gencel [108]. Each threat and its implications are described in the respective sections.

3.6.1. Theoretical validity

Theoretical validity deals with the presence of confounding factors and the ability of the researcher to capture what was intended to capture [108].

A potential threat to this validity is that only the positive results (i.e., what worked well because of the proposed approach) were reported for the evaluated approaches present in the studies. The lack of reporting on the negative results (i.e., what did not work, also known as publication bias) of the reuse approach can handicap the analysis and conclusions of this study. This threat can have a negative impact on the research question two (RQ 2) of this study.

Another threat to the theoretical validity is the small sample size of the industry-evaluated studies. Only 22 studies out of 69 primary studies report an empirical evaluation of a reuse approach. The evaluation of rigor and relevance was conducted on the industry-evaluated studies only. The conclusions are only drawn from the empirically evaluated studies, hence two-third studies (47) were not considered. This smaller sample size, covering various different approaches, may not provide an accurate explanation of the requirement reuse approaches.

The classification of the reuse approaches and artefacts was based on the authors understanding and comparison with the existing literature. The classification can be biased since it is based on the reader's perception and evaluation. The review of the study by two independent authors tries to overcome this bias. However, still, the threat to the validity of the classification of reuse approaches and artefacts is present.

The rigor and relevance scoring system by Ivarsson and Gorschek favors case studies (e.g., context, subjects etc.,) with respect to relevance [79]. Hence, experiments scored lower in relevance as compared to case studies during the evaluation. The usage of this scoring system can have an impact on the accurate evaluation of studies based on experiments.

3.6.2. Generalizability

Generalizability is the degree to which the results can be generalized to different contexts and settings [108].

The inclusion and exclusion criteria of snowball sampling were flexible and include any peer-reviewed study on requirement reuse approach regardless of empirical evaluation. The results obtained from the sixty-nine primary studies generally reflect the requirements reuse approaches present in the literature. These sixty-nine studies contain few studies that are similar to each other but these studies were made part of this review in order to facilitate the reproducibility i.e. snowball sampling depends on the reference list and citations. However, the smaller set of empirically evaluated studies does not fully cover all the requirement reuse approaches. Hence, there is a risk that this validity threat is present.

3.6.3. Objectivity

Objectivity is concerned with the accurate description of objective / subjective truth [108]. Multiple steps were performed to reduce this threat.

The foremost threat to the validity of the study comes from the including and excluding of the studies during the snowball sampling. The bias could lead to the exclusion of the studies that should have been included. This problem was addressed by a two-prong strategy (i) by developing formal inclusion and exclusion criteria and (ii) a reviewer (one of the authors) reviewed a random selection of studies that were excluded and included. The inclusion /exclusion criteria were evaluated jointly by two authors and the third author provided feedback later. Based on the feedback the criteria were updated. Moreover, for the studies that were not clear from the title, their abstracts and conclusions were read before including or excluding these. In few cases, when the abstract was not clear the whole study was read. These steps helped in reducing the risk of missing a study.

Another validity threat is the risk of missing a relevant study. In order to reduce this risk, snowball sampling was complemented with an additional database search. This additional step did not yield any new primary study. Hence, this additional step limited the impact of this threat. Since Scopus covers many of the relevant forums (RE conferences and IEEE digital library), it was selected as a means of sanity-check by identifying any study missed by snowball sampling [111]. The search in Scopus did not reveal any new identified studies and this increases the confidence in the snowball sampling. However, a different database (such as Google Scholar) may have discovered a new set of studies not appearing in our snowballing results or Scopus results.

We attempted to address the threat of misinterpreting the information present in the primary studies by highlighting the concerned parts of the study, extracting the data in the data collection form and then analyzing the results. Two independent reviewers reviewed one-third of the primary studies in order to address this threat. A similar strategy was adopted, involving researchers, to address this threat during rigor and relevance evaluation.

3.6.4. Interpretive validity

The interpretive validity deals with conclusions that are drawn from the evaluated studies. It ensures that the correct conclusions are drawn from the extracted data in an objective manner [108].

In order to address this validity threat, multiple approaches were used such as (i) inclusion and exclusion criteria were formulated for identifying the studies; (ii) a data collection form was developed to extract and analyze the data in a systematic way. The rigor and relevance criteria were developed to systematically evaluate the studies and reduce the bias when evaluating the studies. The bias can lead to different reviewers having a different evaluation for the same criteria. To reduce the chance of bias, two researchers evaluated each industrial study for rigor and relevance. All these steps help in reducing the bias and contextual blindness before drawing a conclusion. However, this threat is still present, i.e., interpretation is subjective to the reviewers' view.

3.6.5. Repeatability

Repeatability ensures that the data collection, analysis and the steps followed during the review are described in detail so that these are repeatable by another researcher [108].

Repeatability is assured by using the snowball sampling guidelines for data collection [85]. These guidelines were strictly followed and each step was documented in the study. For the quality assessment, the guidelines described by Ivarsson and Gorscheck were observed [79]. These details are described in Section 3. These steps ensure that the protocol used in this study can be used to identify similar results and conclusions.

4. Results and analysis

We first present demographic information for the corpus of studies identified. The remainder of the results section is structured according to the research questions formulated in Section 3.

4.1. Demographics

With regard to the demographics, we observed the frequency of studies published over time and the publication types used.

4.1.1. Publications over time

The identified studies were categorized based on the year of

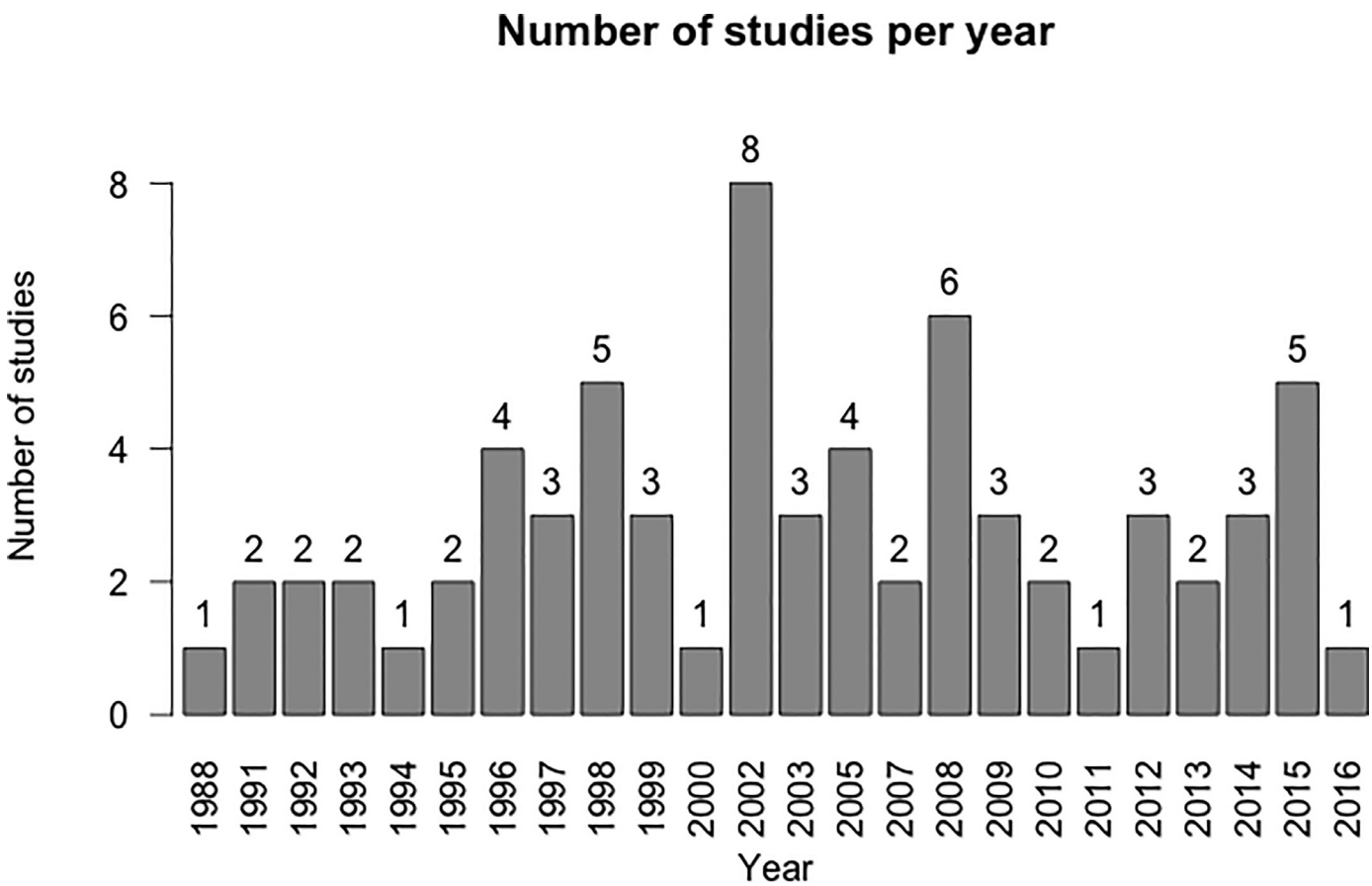

Fig. 2. The distribution over the years.

publication. The earliest identified study was found to be from the year 1988 [28]. The year during which the highest number of studies on requirements reuse approaches were published was 2002. The years of publication were spread from 1988 till 2016. There is a steady trend in the number of studies published on requirements reuse approaches. The frequency by year is shown in Fig. 2.

4.1.2. Publication types

The studies on reuse of requirements were published in a variety of forums and publication types. This study only identified peer-reviewed studies. It was found that conferences are the most widely used publication type. The next most popular type was journal publication. The studies published in the symposium were old studies from the 1990s. Table 7 shows the distribution by publication types.

4.1.3. Publication forums

The analysis of the publication forums of the primary studies reveals that various different forums have published studies on requirements reuse. We have identified forty-five (45) different forums that have published these sixty-nine studies. The well-known requirements engineering conferences such as "International Requirements Engineering Conference" and "REFSQ: Requirements Engineering: Foundation for Software Quality" have published relatively more studies than other forums. Similarly, journals such as "Requirements Engineering" and "Journal of Systems and Software" have published three studies each. A list of the publication forums and the respective number of studies is present in Table 8.

4.2. RQ1: What are the different software requirements reuse approaches present in the literature?

To answer RQ1, we have identified the different requirement reuse approaches, categorized these approaches based on the solution these approaches provide and analyzed their salient characteristics such as artefact used, tool support, repository support and support for software product lines. The studies describing similar approaches and from the same authors were considered as discrete studies. The sections below discuss each of these approaches.

4.2.1. Identified software requirements reuse approaches

Analysis of studies showed that a variety of different approaches is used for enabling reuse of requirements. A brief introduction of the

Table 7
Publication types.

Publication type	No. of studies	Studies ref in Table 7 (* indicates empirical study)
Journal	25	S1*, S2*, S5*, S7, S8*, S9, S11*, S12, S13, S15*, S16, S17*, S19*, S20*, S22*, S25, S28, S36*, S37, S41*, S43, S52, S55, S61*, S67
Conference	34	S3, S4, S6, S10, S14, S18, S21, S23, S24, S30, S31, S32, S33, S34*, S35, S38*, S40, S42*, S44, S45*, S46, S47, S48*, S49, S51, S53, S54*, S56*, S57, S60, S62, S66, S68, S69
Workshops	6	S26*, S27*, S58, S59, S63, S64
Symposium	4	S29, S39, S50, S65

Table 8
Publication forums.

Publication forum	No. of studies	Publication forum	No. of studies
International Requirements Engineering Conference	7	International Conference on Software & Systems Engineering and their Applications	4
REFSQ: Requirements Engineering: Foundation for Software Quality	4	Asia Pacific Software Engineering Conference	3
Journal of Systems and Software	3	Requirements Engineering	3
IEEE Transactions on Software Engineering	2	International Conference on Software Engineering	2
International Conference on Software Reuse	2	International Workshop on Requirements Patterns	2
Knowledge-Based Software Engineering Conference	2	Software Engineering Journal	2
ACM symposium on Applied computing	1	Annals of Software Engineering	1
Applied artificial intelligence	1	Australian Software Engineering Conference	1
Automated Software Engineering	1	Communications of the ACM	1
Computers in Industry	1	Systems engineering	1
First workshop on knowledge reuse	1	IEEE International Symposium on Consumer Electronics	1
IEEE Systems Journal	1	Information and Software Technology	1
Journal of object technology	1	Interacting with Computers	1
Journal of Software: Evolution and Process	1	Journal of Universal Computer Science	1
Journal of Research and Practice in Information Technology	1	International Conference and Workshop on the Engineering of Computer-Based Systems	1
Evaluation and Modeling Methods for Systems Analysis and Development	1	Conference of the Center for Advanced Studies on Collaborative research	1
International Conference on Advanced Information Systems Engineering	1	International Conference on Advances in Computing, Communication and Control	1
International Conference on Human-Computer Interaction	1	International Conference on Information Science and Security	1
International Conference on Software Engineering Advances	1	International Journal of Computer Science and Network Security	1
Data Mining Workshop (ICDMW)	1	Software: Practice and Experience	1
International Symposium on Formal Techniques in Real-Time and Fault-Tolerant Systems	1	international workshop on Software engineering for large-scale multi-agent systems	1
International Symposium and Workshop on Systems Engineering of Computer-Based Systems	1	EURO Mini Conference on Continuous Optimization and Knowledge-Based Technologies	1
Software Reuse: Advances in Software Reusability	1		

Table 9
Approaches used (* indicates empirical study).

Approach	No. of studies	Study ref in Section 7 (* indicates empirical study)
Structuring	16	S3, S5*, S10, S20*, S29, S30, S32, S36*, S38*, S39, S41*, S47, S48*, S49, S50, S62
Matching	12	S7, S8*, S23, S35, S40, S46, S51, S55, S56*, S67, S68, S69
Combination of approaches	9	S1*, S6, S9, S12, S17*, S45*, S58, S59, S63
Analogy	8	S14, S15*, S16, S18, S19*, S28, S53, S65
Model-based Approaches	7	S13, S22*, S24, S31, S37, S54*, S66
Ontology	5	S25, S26*, S27*, S43, S64
Parameterization	4	S34*, S52, S60, S61*
Requirements Traceability	3	S4, S44, S57
Domain Analysis	3	S2*, S11*, S21
Feature Interaction Management	1	S33
Machine Learning Assistant	1	S42*

approaches and the artefacts is also provided. Table 9 lists all the identified approaches and the studies using the approach.

Structuring: In 'Structuring', requirements are stored/saved in a specified organized structures so that these are easier to retrieve when required for reuse. In structuring the requirements are organised (grouped or classified or categorized or arranged) and stored in a specified method. This structuring facilitates in searching and identification of reusable requirements. The studies categorized under this approach can be described as *developing for reuse* [6].

Cybulski et al. (S30) described the classification of requirement texts from different domains and storing in a structured manner to support the cross-domain reuse [30]. They proposed an approach called 'Reuse Assisted Requirements Engineering (RARE)' that helps in structuring the requirements based on keywords and generally capture the functional aspects of requirements. Lim proposed an approach known as 'Scenario Oriented RE-use' (SCORE) based on reuse of scenarios (S48) [48]. In this approach, scenarios are identified, developed and stored in a structured order in reuse repository. Later, the scenarios are extracted and modified for reuse. Keepence et al. (S50) advocate the requirements reuse using the approach known as 'SMART' and structure the requirements into different groups for facilitating the reuse[50]. Lee et al. propose an approach called 'Reusable Requirements Development Process (RRDP)' (S38) [38]. In this approach, reusable requirements are developed and structured under pre-defined categories for the sake of reusing these requirements later. Goldin et al. (S41) suggest a reuse process based on 'project families' that structure the requirements using unique naming [41]. The study narrates the experience from several case studies of projects that have utilised reuse of requirements. Johnson and Harris (S49) describe the structuring of requirements reuse knowledge representation in workspaces and folders to support the reuse of requirements [49]. Melikhova et al. (S29) structured the requirements into 'System Requirements' and 'Implementation requirements' [29]. System requirements are static and do not change while implementation requirements change with different applications. This approach was not validated in the industry though. Pacheco et al. proposed a different approach that uses requirements catalog to support the requirements reuse (S5)[5]. The approach is called 'Requirements reuse model for SRC (RRMSRC)' and it supports requirements reuse activities based on the IEE Std. 830-1998. Hauksdóttir et al. (S20) propose a reusable requirements structure for grouping and reusing similar requirements and the approach was validated in the industry [20]. The study structures the requirements into five groups: Business Requirements, Standards and Laws, Product Properties, Life Phase Requirements and Design Constraints. These groups can contain functional, non-functional requirements, constraints and solution information. According to the authors, this structuring enables the reuse of requirements. Chen et al. (S10) suggest that tightly related requirements should be grouped together to construct a feature using the approach described as 'Requirements Relationship Graphs' [10]. The approach is based on identifying closely related requirements to form a feature model. The relationships between these feature models are

identified and then these feature models are reused. Requirement relationship graphs are constructed from these feature models and later these RRG graphs are reused in other applications.

Another approach, SIREN (S36), which is based on arranging the requirements specification in hierarchal structures for enhancing reuse was suggested by Toval et al. in the context of reusing security requirements[36]. Alexander and Kiedaisch (S59) introduced the concept of recycling instead of reuse but the approach described in the study results in reuse of software requirements [59]. Three levels of structuring are introduced: use-cases, system requirements and the actual system. The recycling approach helps in identifying the suitable reusable candidate among the use-cases. Schmitt and Liggesmeyer proposed a model (S62) for enabling reuse of security requirements [62]. The model structures the requirements under security requirements scope areas and security topics. Each security topic is further described into sub-topics. According to the authors, all these structures can enable reuse in different contexts.

In other structuring based studies, templates are used for the purpose of writing the reusable requirements. Konrad et al. (S32) introduced requirement patterns for eliciting requirements for the reuse and the UML for elaborating the information contained in patterns [32]. The approach mainly used patterns for supporting requirements reuse. An example from the embedded systems is used to demonstrate and explain the approach. Gotzhein et al. (S39) show that the natural language requirements can be structured into requirements patterns for reuse purposes [39]. Saeki in his study (S47) explains how to use-case information is extracted and structured to develop reusable requirements patterns and an example is provided to demonstrate the effectiveness of the approach [47]. A study by Franch et al. (S3) presents a meta-model for the structure of software requirements patterns (SRP), relationships between SRP and their classifications [3]. The approach is based on storing, processing and retrieving the requirements in a structured manner.

Matching: In matching the reusable requirements are obtained by comparing and matching with the existing requirements, using different abstraction levels and approaches such as conceptual graphs, lexical matching etc. The approaches belonging to this category are focused on *developing with reuse* [6].

Blok and Cybulski (S46) used lexical and semantic matching to identify and reuse the requirements represented in UML [46]. This matching is carried out during the domain analysis to identify the reusable parts. The lexical matching is used when searching for UML artefact in many different domain models. The semantic matching is used to identify the UML artefacts when using only one domain model. Paydar et al. (S7) proposed to use matching of UMLs to identify the reuse potential and this matching was done by using a metric for measuring the similarity of UML-based use-cases [7]. Niu et al. proposed requirements reuse approach based on the lexical affinity to retrieve the reusable requirements (S8) [8]. The study introduces the concept of functional requirements profiles (FRP). These FRPs contain high information value related to the functionality present in a document. Information retrieval techniques are used to characterise the text document and extracts valuable information. Lexical-affinity is used to extract conceptual affinity between documents. The study only describes the extraction part of requirement reuse, without discussing the developed with reuse part. In a study on requirements engineering, Castano et al. (S40) describe requirements in the form of conceptual components that facilitate the requirements reuse by matching these conceptual components[40].

Perednikas proposed requirements reuse approach that works by forecasting of user needs (S35). Matching the different set of existing states do the forecasting and the approach is explained with the help of mathematical equations [35]. Ryan et al. used conceptual graphs to represent the requirements in terms of 'objects' and 'messages'. The study (S23) relies on a tool known as 'ReqColl' that uses a matching mechanism to identify the similar cases for reuse [23]. Sutcliffe transforms the use-case scenarios into a model and then compares the model with the library of existing models (S55). The requirements from the matching model are reused to develop new requirements [55]. Biddle et al. proposed an approach (S56) for scenario-based requirements engineering [56]. The approach derives scenarios from the use-cases and suggests the reuse cases by matching with existing scenarios. Periyasamy and Chidambaram used the language Z to represent the specifications (S51)[51]. Each requirement in the specification can be represented by a schema that contains a unique identification and signature. The reuse of requirements can take place if two signatures are matching with each other. The study also describes an algorithm that helps in the implementation of this requirement reuse approach. The approach (S67), "Requirements Reuse Model for Software Requirements Catalog" (RRMSRC), suggests the activities and a method that enable requirements to reuse in requirements catalog [67]. They propose that searching and identification activity can help in maximising the effectiveness of requirements reuse. They suggested a method that utilizes matching to identify the requirements for reuse. Later, these requirements are adapted for requirements with reuse. In another approach (S68) features are extracted from the text requirements and according to the authors, these features can support the reuse of requirements [68]. The high-frequency words are extracted from the documents describing the natural language requirements and based on the frequency of the words the features are identified for reuse purposes. The study describes the method to develop the artefacts for reuse but does not demonstrate how these newly extracted features will be reused. In the second study by Bakar et al. (S69), they demonstrated a reuse approach based on extracting frequently occurring words in documents and used the statistical analysis, based on the distance measure, to identify similar requirements [69].

Combination of approaches: This approach contains studies that use a combination of different approaches for supporting the reuse of requirements e.g., structuring for storing the reusable requirements and matching for identifying the reusable requirements.

Chou et al. retrieved specifications based on behavior described by these specifications i.e., by matching functions and data in the existing specifications (S1, S12) [1,12]. These specifications are classified and stored in the repository before reuse. For retrieval purposes, matching of behavior with the required behavior is used. A similar study, by Benitti and da Silva (S17), utilizes 'catalog of patterns' and the traceability between requirements to describe requirements reuse approach [17]. A study by Mansion et al. (S45) describes requirements reuse approach that uses a combination of application family, a domain model and variable requirements to introduce reusability. The approach is called 'MARM' and facilitates developing for reuse with help of a model and generating new requirements from the set of reusable requirements [45]. Naish and Zhao classify and structure the requirement patterns for the purpose of reuse (S58) [58]. The approach first utilizes the patterns to capture requirements and later classifies these patterns so that they are easy to retrieve for the purpose of reuse. The approach proposed by Dehlinger and Lutz (S63) used a combination of structuring and parameterization [63]. They divide the requirements into two types: the role schema (static part of requirements) and the varying parts (the changing part). Later, the reusability is promoted through parameterization of the varying parts. A study by Sindre et al. focuses on reuse of security requirements by misuse case analysis by providing a meta-model that is used to link the threats with reusable security requirements (S6) [6]. The study distinguishes between two aspects of reuse: developed for reuse and developed with reuse. The requirements are stored in a structured manner and for the identification purposes, matching is utilised to find similar requirements. Lim proposed an approach (S9) based on identifying the variable and static part of requirements and using the parameters to reuse requirements [9]. Domain analysis is utilised to identify the reusable artefacts in avionics domain. Later, a domain-engineering phase is used to package the reusable artefact for reuse purposes.

Analogy: The analogy is used to reason on the similarity of different things by using inference. Several studies have used analogy as an approach to identify the similar cases for reuse of requirements. Generally, these studies map the already defined requirements over the new required requirements using analogical inference. The studies under this category can be used to *develop with reuse* [6].

A study by Massonet et al. (S14) suggests an approach for completing the partially completed specification as opposed to starting from scratch. It requires the use of KAOS language to specify requirements before these can be reused based on analogy[14]. Spanoudakis (S15) uses a computational model to detect the analogies and describes specifications in 'Telos' language [15]. Maiden in his two studies (S16, S19) described analogy as an approach for the reuse of requirements and provides examples as guidelines for the proposed approach [16,19]. However, the articles have limited discussion on retrieval of reusable requirements. The second study (S19) proposes a method to identify and customise a reusable artefact and the study claims that human involvement is necessary for the decision making of the reuse process. A short description of the required artefact is categorized using existing models. Later, a tool supports the software engineers to retrieve the similar models by using different analogies of a model. Finkelstein (S28) describes the application of analogical reasoning on a requirements formatting (known as CORE) in order to facilitate the requirements reuse [28]. Sutcliffe and Maiden proposed (S53) that analogical reasoning could be used to reuse artefacts between different domains [53]. They explained their approach with the help of examples from two different domains. Each domain was mapped into a set of classes, their relationships and the unique identifier [51]. The set of classes from different domains can be combined to form new systems and the unique identifier can be used to retrieve the reusable components. The study, by Wild et al., proposes to reuse artefacts based on formal specification languages (S18) [18]. A domain analysis is performed to identify the requirements and these requirements are then contained in "issue space". Each issue in the issue space represents a requirement and all it's possible outcomes, using Z language. Analogical matching of these issues (in different issue spaces) helps in identifying the reusable requirements. An approach proposed by Chiang and Neubart (S65) revolves around a specification language, known as TUG, which enables the user to specify the specifications and their relationship in a way that can help in reuse. Later, an analogy was used to identify the reusable artefact [65].

Model-based approaches: The main objective of this approach is to understand the reusable parts of requirements by modeling the requirements. The model-based approaches work by analyzing and developing a model of requirements that is later used for identifying reuse cases. These models can be based on mathematics, on features, on tasks or on domain specific languages. A study by Moros et al. (S24) suggested using 'Requirements Engineering Meta Model (REMM)' to facilitate the reuse of requirements [24]. The model is extended to use parameterized requirements for enhancing the reusability of reusable requirements. However, the requirements reuse approach uses the requirements modeling to enable reuse, hence the study is classified under model base approaches. A feature model represents the features and their dependencies in a product line. Gomaa (S37) suggested to use feature modeling and differentiates between the features in a domain as a kernel (required) and optional [37]. Based on this distinction the optional features are considered for reuse. Another study by, Eriksson et al. (S22), discusses the requirement reuse in the product line context by modeling the variability of requirements and combining it with feature models [22]. These feature models helped in providing a common understanding and view of different product variants in product lines. Any new product contains a new instance of feature model based on the viewpoint of that product. Lopez et al. (S66) addressed the reuse of requirements in a domain where the concept of product families have not been used yet [66]. They proposed an approach in which identified requirements of a domain are converted into semi-formal diagrams. These diagrams are integrated into meta-model and stored for future reuse opportunities.

Task models are a useful way to describe the interactive systems by detailing the activities required to perform a certain task. Montabert et al. proposed two approaches (S13, S54), based on task modeling, to reuse requirements[13,54]. In the first study, parameter based task modeling was introduced to facilitate the requirements reuse process. A feasibility study explains the concepts behind the proposed approach (S13) [13]. The second study described the hierarchical task analysis approach that claims to support the requirements reuse (S54) [54]. The approach proposed by Schmitz et al. used 'i star' modeling language with control systems concepts to facilitate reuse of requirements [31]. The study (S31) investigates different aspects of requirements engineering using this approach (e.g., requirement processes) and it is not limited to requirements reuse.

Ontology-based approach: Ontology-based approaches suggest that requirements engineering is a knowledge intensive process thus usage of knowledge-based approaches will enhance the requirements reuse.

Karats et al. (S26) suggest an ontology-based requirements reuse approach applicable to requirements in software product lines [26]. The approach is demonstrated with the help of a case study. Velasco et al. (S25) proposed a combination of risk analysis ontology and requirements ontology for reusing the security requirements [25]. In another study on reuse of security requirements, Daramola et al. (S27) used the ontology-based approach to achieve the requirements reuse [27]. Marrero et al. (S64) introduce an ontology-based approach called REOntology that works as reuse enabling system [64]. The approach is based on three main modules: requirements description, indexing, and retrieval. The controlled vocabulary is used to extract the meaning of requirements mentioned in the requirements description. Indexing helps in associating the requirements with the controlled vocabulary while retrieval mechanism helps in extraction of reusable requirements. An ontology-based approach (S43) was proposed by Bonilla-Morales et al. for enabling reuse of use-cases [43]. A tool that supports the reuse of use-case diagrams, constructed in UML, implements the approach proposed by Bonilla-Morales et al. The use-case diagrams are stored in ontological representation (semantic web language) in a repository. Users of that repository can query it to identify the diagrams that they need for reuse purposes. The web ontology language enables the user to represent, retrieve and use the reusable use-case diagrams designed in UML.

Parameterization: In 'Parameterization' the requirements consist of two parts, a static part, and a variable part. The variable part contained values from the parameters and these parameter values describe the actual behavior of the system.

Heumesser et al. described the use of parameterized requirements, based on dependent and independent parts, for reuse purposes (S60) [60]. They termed the reuse as 'recycling of requirements'. Mannion et al. support requirements reuse by developing a model that contains parameterized discriminants [61]. The study (S61) is conducted in the context of software product lines and provides an example to explain the concepts. Another study, by Firesmith, introduces parameterized templates for reusing the security requirements. It further proposes an asset-based approach to identify the appropriate parameter (S52) [52]. Hauksdottir et al. (S34) propose two approaches to reuse the requirements; the first approach requires that an analysis should be performed on the requirements to identify the common and variable parts [34] and parameterization is used for variable parts to support the reuse in other projects. In the second approach, they suggested to analyze the requirements and develop a common model from different requirements, later this model should be parameterized to enable reuse.

Requirements Traceability: Requirements traceability provides the link between different requirements for the purpose of tracking and management. This tracking between requirements helps in identifying the reusable requirements.

Monzon proposed an approach (S4) based on the assumption that in

a product family the requirements are traceable between different products [4]. Each time a new product is developed traceability is used to identify the reusable requirements. Pooley et al. (S44) attempt to facilitate reusability by a combination of traceability of the requirements in requirement management tool and code in the source control systems [44]. The approach relies on the tool support (versioning control software) to identify the reusable artefact. A study by Von Knethen et al. (S57) combines abstraction and traceability to document the commonalities among requirements and establish the relationships of requirements and this combination resulted in the reuse of requirements [57].

Domain Analysis: The domain analysis approach was used in the product line context to develop requirements that can be used in similar applications.

Lim (S2) suggested a domain analysis based requirement reuse approach called 'Family of Requirements (FORE)', that develops a family of reusable requirements that can be used in several similar products [2]. The approach is spread over the project lifecycle and involves various project stakeholders. Moon et al. (S11) proposed a CASE environment known as DREAM for commonality and variability analysis of software requirements in a product line context [11]. The approach is used for developing reusable domain specific requirements to be used in software product line. The study introduced a term 'Primitive Requirements' (PR) that acts as a building block for future requirements. The approach utilised these PRs to develop for reuse. Later, these PRs are used to develop domain use-case model. The study describes the approach with a case study on e-Travel systems. Gross et al. (S21) proposed a reuse driven systematic process, known as 'Reuse-Driven Software Engineering Business' that uses domain modeling to identify the variability and support the reusability of requirements [21]. The study combines the feature models and reuse-driven software engineering business models to enable the reuse of the features.

Feature interaction management: Shehata et al. (S33) propose that feature interaction management is key to understanding the requirements reuse. They propose a 'Conceptual Process Framework' for supporting the requirements reuse [33]. The approach uses interaction management in the context of requirements reuse to facilitate reusability. The approach is dependent on the interaction and control of features and does not belong to already existing reuse approaches.

Machine learning assistant: Machine learning, using an automated graph-based relational tool, was used to identify the reusable requirements by Woo et al. (S42) [42]. The user inputs the requirements and interactions as an input to the tool and the tool suggests the most relevant use-cases for reuse.

4.2.2. Characteristics: forms of reusable requirements

The approaches used for reusing requirements provide diverse solutions and these approaches are implemented at various abstraction levels. Similarly, these approaches require different artefacts and representation of requirements for reuse purposes. Each approach used a different abstraction level of requirement artefacts. In this section, a reusable artefact is considered to be the input to requirements reuse approach, regardless of the manipulation performed on the artefact when the approach is executed. The synthesis of the studies identified the artefacts shown in Table 10. The description of each identified artefact type is given below:

Abstract or Undefined: The artefacts classified as abstract are those in which the reuse approach does not impose any limitations regarding the format of the requirements. The methods described in such studies did not describe a specific requirement format to support the requirement reuse. In limited cases, studies do not contain information about the artefact that can be reused; instead, the studies discussed reuse approach only. In this section, we have described studies that do not explicitly discuss the type of reusable artefact.

Maiden (S16, S19) in his two studies described analogy as an approach to reuse requirements specifications in an artefact independent way [16,19]. The emphasis of the studies is on the explanation of reuse approach and these studies do not discuss the artefact to reuse. Hauksdóttir et al. described the structuring of the requirements (S34), without mentioning the format and type of the requirements, to support the reuse of the requirements [34]. The approach proposed by Perednikas (S35) is described with the help of mathematical equations and does not mention the type of the artefact [35]. Another approach proposed by von Knethen et al. (S57) described 'conceptual system model (CSM)' as the fundamental unit on which the approach was based on [57]. Fundamentally a CSM is a representation of the relationship between different requirements documents and the types of the entities present in a specific domain. Schmitt and Liggesmeyer (S62) described their approach without utilising any artefact format [62]. The approach revolves around security requirements specification without mentioning the format and abstraction level of the specification. The approach (S53) proposed by Sutcliffe and Maiden is described at high abstract level and in a way that it is independent of any specific reusable artefact [53]. The approach uses analogical reasoning to identify the reusable artefact. A study by Johnson and Harris (S49) describes the requirements reuse approach without discussing the artefact that can be reused [49]. The approach described the structure and hierarchal organisation of artefacts for the purpose of reuse without elaborating on the details of the artefact. A study on reuse-formatted requirements by Finkelstein (S28) used the flow chart transactions as an input to the proposed approach [28]. These transactions described the changes in the state of a given system. An approach by Shehata et al. (33) divided the requirements into following groups: domain-specific requirements, generic requirements, and domain requirements frameworks [33]. These are the lowest levels of abstraction on which the approach is further elaborated and the study does not mention any specific format of these types of requirements.

Gomaa, in his study on reusable requirements and architecture (S37), described an approach that can be applied to any type of requirement artefact [37]. The format of the requirement or any guidelines to structure the requirements were not present in the study. While describing the reuse of requirements using traceability of code present in the versioning control software (S44), Pooley and Warren did not impose any restriction on the type of requirement artefact [44]. The approach was limited to retrieval of the reusable artefacts by analyzing the source code. The artefact was described by Castano et al. (S40) as the application independent specifications derived from many different application dependent specifications and contains the guidelines for

Table 10
Forms of reusable requirements.

Artefact type	No. of studies	Study ref in Section 7 (* indicates empirical study)
Abstract (or Undefined artefact)	20	S1*, S2*, S11*, S12, S16, S19*, S20*, S23, S28, S33, S34*, S35, S37, S40, S41*, S44, S49, S53, S57, S62
Textual Form	19	S4, S5*, S8*, S9, S13, S25, S26*, S29, S30, S45*, S48*, S50, S54*, S61*, S63, S64, S67, S68, S69
Template	10	S3, S6, S17*, S27*, S32, S36*, S47, S52, S58, S60
Use-case	8	S7, S21, S42*, S43, S46, S55, S56*, S59
Modeling Language based artefacts	6	S14, S15*, S24, S31, S39, S66
Formal Specification	3	S18, S51, S65
Feature	3	S10, S22*, S38*

supporting the reuse of these conceptual units [40]. However, the approach was brief and example scenarios were not elaborative enough to explain the usage of this artefact. Goldin and Berry (S41) described the reuse experiences from several case-studies from the different project [41]. However, the study is described at a high level and no details on the type of reusable artefact are provided. A conceptual graph is an abstract collection of nodes, described as 'concept' and 'relation' nodes linked with each other [23]. These graphs are used to describe the systems and relations. Ryan et al. (S23) developed a methodology to reuse the conceptual graphs representing the requirements [23].

Shih–Chien et al. (S1, S12) demonstrated an approach that classifies the requirements based on behavior and used this approach to facilitate the requirements reuse [1,12]. Lim (S2) used text-based product analysis to demonstrate his requirement reuse approach[2] but the requirements artefact was not clearly described. Moon et al. introduced the term "Primitive Requirement" that described an abstract concept as a minimal building block that can be used to construct a requirement [11]. The authors do not provide the details on the structure, content or format of these primitive requirements. In another study on reuse of requirements (S20), Hauksdóttir et al. described high-level structuring of requirement groups but do not discuss the details of the format and type of the requirement artefact [20].

Textual form: The text-based artefacts include artefacts written in the natural language or in descriptive text. Most of the identified studies focus on reusing text-based artefacts. It is worth mentioning that studies often transform text-based documents into different forms (formal specification, scenarios etc.,) but the studies listed in this section work primarily with the text-based artefacts. A brief introduction of the types of textual artefacts, used by the studies in this category, is given in this section.

Montabert et al. (S54) provide a task, in the form textual format, as an input to the task based requirements reuse approach [54]. Dehlinger and Lutz (S63) used the text-based requirements to enable parametrization of variable parts, reflecting an event that an agent can perform [63]. These variable parts were described in the form of textual form. Lim proposed a scenario-based approach (S48) to complement the requirements reuse approaches [48]. Montabert et al. (S13) describe the use of scenarios in task models to support requirements reuse [13]. Monzon in his study on reuse of requirements (S4) in product families used text requirements[4]. The requirements that were written using IEEE Std. 830-1998 (textual form) were used by Pacheco et al. in their study when using catalogs (S5) [5]. The tool developed by Mannion et al. (known as TRAM in S45) use the text-based requirements as input [45] and the text-based artefact was used in another study by the same author (S61) [61]. Keepence et al. (S50) described the reuse approach using text-based requirements [50]. The approach described by Marrero et al. in their study (S64) utilizes the requirements written in textual form and process these requirements to comply with the REOntology method[64]. The task describes a scenario that can enable reuse in different use-case flows. In a study on reusing of functional requirement profiles, Niu et al. (S8) used the text-based functional requirements to explain their reuse approach [8]. Lam used text-based requirements (S9) to introduce his reuse approach [9]. The approach is applied to different types of text-based artefacts. Velasco et al. (S25) used the text from the natural language to demonstrate his approach to reuse security requirements [25]. The reusable security requirements are extracted from the pre-defined information about different levels of risks. Karatas et al. (S26) used the text-based requirement specification document in order to apply their proposed approach to requirements reuse [26]. Not enough details about the format of this specification document are present in the study. Melikhova et al. (S29) classified the requirements into two types: system requirements (stable) and implementation requirements (varying) [29]. Both types of requirements were considered as text requirements by the authors while explaining the proposed requirement reuse approach. Cybulski and Reed (S30) based their approach on "informal requirement texts" and proposed the method through which these informal requirement texts can be used across several projects [30]. Bakar et al. proposed two different requirements reuse approaches (S68, S69) in their two studies[68,69]. Both approaches used text-based requirements for supporting the reuse of requirements, these textual requirements were written in natural language. Pacheco et al. (S67) described a requirements catalog approach that uses the software requirement specifications (SRS) based on IEEE Std. 830-1998 [67]. These requirements are textual in essence but the approach did not discuss details of the artefact.

Template: A template is an aide that provides guidelines for writing requirements in a specified format. The format can be of any type (text, ordered steps etc.,) but written using a specified format.

Benitti and Silva proposed a reuse approach (S17) that utilizes traceability and structuring of templates (described as patterns). These templates contain information that is vital for enabling the requirements reuse [17] such as objective, context, problem, description and an example. These templates are later used for supporting the reuse between different cases. A study by Franch et al. (S3) used requirements patterns written in the natural language to demonstrate the reusability of requirements [3]. Daramola et al. (S27) used patterns, with help of ontology-based approach, to support the reuse of requirements [27]. Firesmith introduced a text template (a paragraph of text) with configurable values that can help in the reuse of the template for different scenarios (S52) [52]. The template provided was specific for reuse of security requirements and does not consider reuse of other requirements. An example was provided to demonstrate the use of this text template. Heumesser and Houdek (S60) [60] provided a documented structure to identify and write the independent and dependent requirements in a pre-defined template. The template contains details such as context, relationships, interfaces and description related to a requirement. Naish and Zhao used the patterns (in the specified template) to explain their approach to reuse of requirement patterns [58]. Saeki uses use-case templates in their study (S47) on reuse of use-case patterns[47]. The study proposes a template that can be used to document a use-case; later that template is used to support the reuse. Toval et al. (S36) in their study on reuse of security requirements used a template for requirements specification based on IEEE Std 830-1998. [36]. These templates are arranged in a hierarchal manner to support the retrieval part of requirements reuse. Sindre et al. (S6) in their study on reuse of security requirements proposed a template, known as a misuse case, that can help in capturing, identification and reusing of security requirements [6]. UML-based templates were introduced by Konrad and Cheng (32) in their study on requirements in embedded systems [32]. These templates were designed in UML and implement different patterns that are commonly used in embedded systems. Naish and Zhao (S58) based their study on the reuse of requirements pattern where they define the pattern as a template that contains information specified format [58]. The study does not discuss the format of these templates but discusses the classification and retrieval of these templates.

Use-case: Jacobson described use-case as a way of using a system to achieve a particular goal for a particular user [105]. Use-cases were identified as a popular way to facilitate reuse of requirements in several studies. Alexander et al. (S59) formulated the use-cases from the specification and suggest that 'recycling' of these use-cases is an effective way to develop reusable requirements [59]. Woo et al. (S42) proposed a reuse of use-cases with the help of a tool based on machine learning [42]. Griss et al. (S21) introduced a use-case driven systematic reuse process, which transforms the use-cases into object models before supporting requirements reuse [21]. Sutcliffe et al. (S55) extracted scenarios from use-cases and applied their approach on these scenarios [55]. They defined a scenario as one possible path in a use-case. Biddl et al. (S56) used use-cases to enable the reuse of scenarios as a method of reuse of requirements [56]. A tool is used to capture the scenarios in the form of use-cases. Blok and Cybulski (S46) used the similarity of use-case flows acts as the matching factor to support reuse of use-cases

[46]. The use-case diagrams in UML are used in a study (S43) reported by Bonilla-Morales et al. [43]. The approach is based on reusing use-case diagrams that are assisted by web ontology language (OWL). Paydar et al. in their study (S7) on reuse of functional requirements used UML based activity diagram and use-cases [7].

Modeling language based artefacts: Unified Modeling Language and its dialects (SysML) have been used by several studies to support requirements reuse. A study by Gotzhein et al. (S39) used 'SysML' based artefact to demonstrate the reuse approach[39]. A meta-model was developed using 'SysML' to enable requirements reuse in a study by Moros et al. (S24) [24]. The KAOS language is requirements capturing language used by Massonet et al. in their study (S14) on requirements reuse[14]. The approach described in the study reused the artefacts written in KAOS. The 'i star' language was used by Schmitz et al. to facilitate the reuse of requirements (S31) [31]. Telos is a modeling language to support the development of information systems. Spanoudakis et al. (S15) considered a representation of specifications in Telos language as a reusable artefact [15]. Lopez et al. (S66) proposed that requirements expressed in the form of graphical diagrams are easier to express, understand and reuse [66]. Their study (S66) proposed an approach that used these diagrams to support requirements reuse. The reusable artefact used in the approach is based on the "Coloured Petri net" (CPN) mathematical modeling language.

Formal Specification: Formal specifications are the mathematical representation of software requirements that are used to describe the system and facilitate in understanding the requirements. Three studies (S18, S51, S65) were identified that used formal specification based artefact for enabling the reuse [18,51,65]. Wild et al. (S18) and Periyasamy et al. (S51) used Z notation for the demonstration of reuse approach [18,51]. TUG is a formal specification language that was used by Chiang and Neubart (S65) to specify the reusable requirements [65]. According to the authors, this formal specification language can support the reuse of requirements because it contains useful features such as polymorphism, overloading, and the language is easier to write and read. Their approach used this language to perform analogical reasoning to identify reusable requirements.

Feature: A feature is a unit of the system that represents a given functionality [104]. Chen et al. (S10) used features to construct feature models and cluster these feature models for the purpose of reuse [10]. Lee et al. (S38) in their proposed approach used the feature as reusable requirements [38]. They described the feature as a form of functional requirements only. The non-functional requirements were not included in their definition of a feature. Eriksson et al. (S22) used feature models to manage variability in product line requirements [22]. These feature models enable large-scale reuse in product lines.

4.2.3. Characteristics: tool support

CASE tools are used in software development for the design and implementation of software artefacts in different stages of the software project lifecycle [102]. The studies, identified using snowball sampling, have described the usage of a tool for assisting in the requirements reuse. However, it was noted that not all studies describe the complete working of the tool; many studies mentioned the tools but do not describe the details of the tool. It was noted that many of the evaluated industrial studies mention that tool support was used during the evaluation of the proposed approach. Table 11 lists the studies describing the tool support, the studies not using the tool support, the studies mentioning the tool support but not describe the tool and studies that mention the tool support as future work.

4.2.4. Characteristics: repositories

In order to store and retrieve the reusable requirements, repositories/libraries are described in several studies. These repositories assist in searching and identifying reusable requirements [103]. Table 12 lists the studies describing requirements reuse repositories. However, a detailed discussion on the impact of these repositories is missing in all the identified studies. The evaluated industrial studies mention the use of repositories but do not reveal the details of the repository. The studies present in Table 12 are the ones that have used repositories when demonstrating the requirements reuse approach.

4.2.5. Characteristics: support for software product lines

The fundamental part of Software product lines is to reuse artefacts, during different stages of product development in a planned manner. Sixteen studies discussed the role of requirements reuse in the context of software product lines and half of these studies were evaluated in the industry. The studies discussing the impact of reuse of requirements on product lines are listed in Table 12.

4.3. RQ2: how have the requirements reuse approaches identified in RQ1 been evaluated, and what was the outcome?

To answer RQ2, we looked at the research method used, and also focused on which approaches and related characteristics have been the focus of industrial evaluations. Furthermore, the results of the evaluations for case studies as well as experimental studies are described.

4.3.1. Research methods used

The reuse approaches described by the primary studies were explained with help of industrial or academic examples. This provides an indication if the study was validated in the industry or not. Most of the studies used an example application to describe the reuse studies. These examples were hypothetical examples and these studies revolved around an application that reuses requirements based on the proposed requirements reuse approach. Industrial Case Studies were the second most used research method. Table 13 shows the respective distribution of research methods used in the identified studies.

4.3.2. Focus of evaluations

We looked at the focus of industrial evaluations from three perspectives: reuse approaches, forms of reusable requirements, and trends over time.

Industrial evaluation vs. Reuse approaches: We classified all the identified studies into different reuse categories, listed in Table 9. The empirical studies show that only a subset of the reuse approaches have been evaluated in the industry, with the main focus being on reuse through structuring. Table 14 list the studies and the corresponding approaches used.

Industrial evaluation vs. Forms of reusable requirements: Seven forms of reusable requirements were identified that are used by the sixty-nine

Table 11
Characteristic: tool support.

Characteristic	No. of studies	Studies (* indicates empirical study)
Studies *with* tool support and describing how the tool works.	34	S1*, S4, S5*, S9, S10, S11*, S12, S13, S15*, S17*, S18, S19*, S20*, S21, S22*, S23, S24, S26*, S27*, S28, S36*, S41*, S43, S44, S45*, S53, S54*, S55, S56*, S60, S64, S65, S66, S67
Studies *without* tool support.	17	S8*, S14, S25, S29, S32, S33, S35, S39, S40, S46, S47, S48*, S51, S52, S57, S62, S69
Studies mentioning a tool but not describing how the tool works.	15	S2*, S3, S7, S16, S30, S31, S34*, S37, S38*, S42*, S49, S59, S61*, S63, S68
Studies mentioning tool support as *future work*	3	S6, S50, S58

Table 12
Characteristics: repositories & characteristics: support for software product lines.

Characteristic	No. of studies mentioning characteristic	List of studies ***with*** support for characteristic. (* indicates empirical study)	List of studies ***without*** support for characteristic. (* indicates empirical study)
Repositories	31	S1*, S3, S4, S5*, S6, S7, S11*, S12, S13, S14, S17*, S20*, S21, S24, S26*, S27*, S30, S34*, S36*, S37, S40, S41*, S43, S44, S48*, S51, S53, S54*, S55, S66, S67	S2*, S8*, S9, S10, S15*, S16, S18, S19*, S22*, S23, S25, S28, S29, S31, S32, S33, S35, S38*, S39, S42* , S45*, S46, S47, S49, S50, S52, S56*, S57, S58, S59, S60, S61*, S62, S63, S64, S65, S68, S69
Support for Software Product Lines	16	S2*, S4, S8*, S10, S11*, S22*, S26*, S31, S34*, S41*, S57, S60, S61*, S63, S66, S69	S1*, S3, S5*, S6, S7, S9, S12, S13, S14, S15*, S16, S17*, S18, S19*, S20*, S21, S23, S24, S25, S27*, S28, S29, S30, S32, S33, S35, S36, S37, S38*, S39, S40, S42*, S43, S44, S45*, S46, S47, S48*, S49, S50, S51, S52, S53, S54*, S55, S56*, S58, S59, S62, S64, S65, S67, S68

Table 13
Research methodologies used. (* indicates empirical study).

Research methodology	No. of studies
Solution with Example Applications	41
Case Study *	15
Experiment	7
Lessons Learned	6

primary studies, present in Table 10. The evaluation results point to the presence of industrial evaluation with regard to the majority of forms of requirements. Seven of the evaluated industrial studies used text-based requirements in their approaches (S5, S54, S8, S61, S48, S45, S26). Seven studies (S1, S2, S11, S19, S20, S41, S34) were described in a way that these were not dependent on any specific form of requirement or they did not describe the reusable artifact in detail. Only the "Formal Specification" based studies were not evaluated in the industry. Table 14 shows the forms of requirements utilised in the industrial evaluations.

4.3.3. Case study results

The analysis and evaluation are performed on the identified case studies regarding the study objectives, approach, data collection methods, as well as measured variables and other observations. The results of the case study based studies are described in Table 15. The main aspects of the analysis are described below:

Measured variables: During the analysis of the studies, it was evident that these studies do not have uniform definitions (or understanding) of the terminologies and the variables. However, analysis of each identified studies showed that there are few common variables present in each study. The most frequently used variable in the identified studies is the degree of reuse i.e., the number of reuse instances. It describes that the focus of the study is on increasing the reuse instances of the reusable requirement. Another measure, for the evaluation of approach, was the suitability of the approach for the software product lines i.e., if the approach used can support the software product lines. The third variable identified from the studies was the development effort i.e., the impact of the approach on reducing the development time spent on the product development. Furthermore, few studies evaluated the impact of the approach on requirement management, impact on risks associated with writing new requirements and the impact on the quality of the requirements.

Positive results: The impact of the above-mentioned variables is analyzed from the studies. It was found that all these studies describe only the positive impact of the approaches. These studies do not describe the negative impact of the reuse approach on the software project or product development. Only in one study the findings were considered inconclusive (S38) and impact of the approach on the measured variables was not possible to analyze.

Methodology All the identified case studies used document examination as the method for data collection. This is understandable since the software requirements are documented in the textual form. It was not possible to identify if the document examination was done with the help of manual reading or by some other mechanism. The reuse approaches utilize human subjects and documents for analyzing the effectiveness of the approach, thus using qualitative and quantitative assessment.

Tool integration: In many cases, the requirement reuse approaches were integrated into tools and the evaluation was conducted using the developed tool. The tools help in the collection, analysis, and generation of reusable requirements. An overview of how the tool worked as well as the snapshots of tool were provided by the studies. These tools automate the steps defined by the approach to support the requirements reuse. More than half of the case studies (S11, S19, S20, S22, S26, S36, S41, S45, S56) used the help of a tool to support requirement reuse. A separate study is required to analyze and compare the working of these tools.

Generalization of results: The objectives of the identified studies

Table 14
Focus of evaluation.

Evaluated industrial studies by approach		Evaluated industrial studies by type of artefact	
Approach	Studies (in Section 7)	Artefact type	Studies (in Section 7)
Structuring	S5, S20, S48, S41, S38, S36	Textual Form	S5, S8, S26, S54, S45, S48, S61,
Combination of Approaches	S1, S17, S45	Abstract or Undefined	S1, S2, S11, S19, S20, S41, S34
Matching	S8, S56	Template	S17, S27, S36
Ontology	S27, S26	Use Case	S42,S56
Domain Analysis	S2, S11	Feature	S22, S38
Parameterization	S34, S61	Modeling Language based artefacts	S15
Model-based Approaches	S22, S54	Formal Specification	-
Analogy	S15, S19		
Machine Learning Assistant	S42		
Requirements Traceability	–		
Feature Interaction Management	–		

Table 15
Analysis of case studies, (+) refers to a positive result, (−) to a negative result, and (~) to an inconclusive result for the studied variables.

Study	objective	Approach	Method/ Data collection	Measured variables	Other observations
S2	To propose and evaluate a method for early reuse in aircraft engine control system.	The "FORE" (Family Of Requirements) approach.	Case study, Document examination	SPL suitability (+), Degree of Reuse (+)	The study discusses the impact of approach on development and organizational processes.
S8	To provide a light weight approach for automated extraction of reusable requirements	An approach that develop requirements for reuse by extracting the functional requirements profiles (FRP) from natural language documents.	Experiment and Focus Groups, Document examination	SPL suitability (+) , Quality of Requirements(~)	The focus group found the approach to be better and suggested further evaluation before using the approach in the industry.
S11	To propose an approach that increase reusability in software product lines by developing requirements as core asset.	The approach suggests a four step process to develop and generate requirements for reuse.	Case study, Document examination	SPL suitability (+) ,Degree of Reuse (+)	A case environment, DREAM, is used to assist in the proposed methodology.
S19	To investigate the benefits of analogy as a method for reusing requirements specification.	The approach suggested in the study uses a three steps step method to develop and reuse requirements for reuse.	Case study, Document examination	Degree of Reuse (+), Development Effort (+)	The study discusses a case tool, Ira, to implement and describe the proposed approach.
S20	To proposing a software requirement structuring approach to fill the shortcomings of existing approaches.	The new approach suggests to structure the requirements in the groups, having sub-groups, based on the analysis of the product environment viewpoints.	Case study, Document examination	Acceptability (+), Development Effort (+)	A defined criteria and the shortcomings of the existing structuring approaches, were addressed in the new requirement structuring approach.
S22	To propose a requirements reuse approach that uses feature models in order to support reuse in software product lines	The approach uses a method called PLUSS, based on feature modeling to address the variability in different products of a product line.	Case Study, Document Examination, Participant Observation, Interviews	Acceptability (+) , Degree of Reuse (+) ,SPL suitability (+)	The study consists of multiple data collection methods in order to address the research questions.
S26	To propose an ontology-based requirement reuse approach for software product lines.	The approach is applied with help of a tool, OntSRDT, that develops a requirement artefact for the new product in produce line, based on user's input	Case study, Document examination	Degree of Reuse (+) , Quality of Requirements (+)	The study introduces different matrices in order to measure the mentioned variables.
S34	To compare and evaluate the two requirements reuse approaches, proposed in the study	The first approach reused the requirements from the previous project, while the second approach reused the requirements from a pool of requirements forming a joint model.	Case study, Document examination	Degree of Reuse (+), SPL suitability (+)	The first approach is suitable for less mature domain, while second approach is suitable for a stable domain.
S36	To propose a approach for elicitation, reusing and managing software requirements	The proposed approach uses SIREN methodology for describing the reuse of security requirements.	Case study, Document examination	Degree of Reuse (+), Risk Assessment (+)	Security requirements were used to explain the approach.
S38	To propose a systematic requirements reuse process in the embedded software domain.	The proposed approach is built on feature as fundamental unit for reuse of requirements.	Case study, Document examination	Degree of Reuse(~) Requirements management (~)	The approach is based on two phases: (a) identification and formulating requirements for reuse (b) using the reusable requirements
S41	To propose, evaluate and report the results of project family based requirements reuse approach.	The family based approach is based on the products that share considerable number of artefacts, with the possibility of reusing each other's artefacts in future.	Case study and Interviews, Document examination	Degree of Reuse (+) , Development Effort (+)	A detailed industry evaluation, using case study and interviews, was conducted to find the pros and cons of the approach.
S45	To propose a requirements reuse approach based on application of families.	The approach assumes that similarity in the application families can help in reuse of the requirements. The approach consisted of three main parts, a pool of requirements, a domain model and discriminants.	Case study, Document examination	SPL suitability (+)	The study utilizes a tool "TRAM" during the execution. The study extends the already existing MARM model.
S48	To propose a reuse approach based on the reuse of scenarios.	The study discusses the scenario reuse approach and the impact of the approach on the domain and measurements associated with reuse of scenarios.	Case study, Document examination	Degree of Reuse (+ impact), SPL suitability (+)	The study compares the approach with contemporary approaches and suggests different levels of abstractions of scenarios.
S56	To propose a approach for scenario-based requirements engineering.	The approach derives scenarios from the use-cases and suggests the reuse cases by matching with existing scenarios.	Case study, Document examination	Degree of Reuse (+), Requirement Management (+)	A tool, CREWS-SAVRE, is used to implement and demonstrate the approach.
S61	To propose a approach for reuse of requirements in software product line requirements.	The approach uses parameterized discriminants in order to propose a process of capturing product line requirements.	Case study, Document examination	SPL suitability (+)	The study extends the already existing MARM model.

Table 16
Analysis of experiments, (+) refers to a positive result, (−) to a negative result, and (~) to an inconclusive result for the studied variables.

Study	objective	Approach	Method/ Data collection	Measured variables	Other observations
S1	To propose an approach for objected oriented specification reuse.	Specifications with similar behavior (based on semantic meaning) can be considered for reuse.	Experiment, Document examination	Degree of Reuse (+)	The study relies on the semantic similarity to retrieve the potential reusable requirements.
S5	To propose a model for reuse of requirements in Requirements Catalog.	The requirements reuse model for Software Requirement Catalog (RRMSRC)	Experiment, Observation Participant, Document examination	Degree of Reuse (+), Development Effort (+)	Enable requirements reuse in same domain, the model is aimed at small and medium enterprises.
S15	To propose a computational model to facilitate analogy detection.	Specifications can be reused by detecting analogies.	Experiment, CASE Tool	Degree of Reuse (+), Complexity to reuse (+)	The study discusses the similarity analysis conducted by the proposed model with detection by humans.
S17	To propose that reuse of requirements results in greater reuse of other software artefacts such as model, code and tests.	The approach uses combination of requirements patterns, traceability and patterns catalog.	Experiment, Questionnaire	Requirement Management (+), Degree of Reuse (+)	Quantitative and qualitative approaches were used to assess the effectiveness of this approach.
S27	To aid requirements specification generation using pre-defined pattern based templates.	The pre-defined templates can be used to enable the reuse of requirements using an ontology-based approach.	Experiment, Questionnaire	Development Effort (+), Degree of Reuse (+)	A tool, ReqSec, was used in evaluation of this approach to develop software requirements specification.
S42	To propose a approach for assisted reuse of use cases.	A requirements reuse approach based on the semantics of UML and diagrams.	Experiment, Case tool	Degree of Reuse (+)	The approach is illustrated with help of tool called SCENASST(SCENario ASSisTant), utilizing machine learning.
S54	To propose an approach for task analysis for supporting reuse.	Integrate the requirements into task models the enable the reusability of requirements.	Experiment, Questionnaire	Task Modeling (+), Degree of Reuse (+)	The study revolved around a tool, LINK-UP, that is used to design the notification systems through reuse.

were not limited to the requirements reuse and these approaches work at different abstraction levels e.g., a study (S8) used functional requirement profiles while another study is limited to reuse of security requirements (S36) only. These studies do not discuss the applicability of the approaches in different contexts and settings. Therefore, it is a shortcoming present in the identified case studies that generalization of the results is not possible.

4.3.4. Experiment results

The studies based on the experiments are analyzed and evaluated with respect to the study objectives, approach, data collection methods, as well as measured variables and other observations. The results of the experiment based studies are summarized in Table 16. The prominent aspects of the evaluation are described below:

Measured variables: Similar to the variables used in the case studies above, the degree of reuse is the most frequently used aspect that experiments based studies address. This measure describes the success of the reuse approach narrated in the study. The studies, based on experiments, evaluated aspects such as the impact of the approach on development effort, the impact on the complexity to reuse the requirements generated by the described approach and the impact on the task modeling. Each study described the impact as positive on each measured variable. The negative results, such as the resources required to implement the approach, the impact on development process etc., were not described in the studies.

Methodology: The analyzed studies used qualitative and quantitative approaches to assess the usefulness of the proposed reuse approach. The studies used document examination, survey questionnaires, and observations as the methods for data collection.

Tool integration: From the analysis of the experiment based studies it was evident that these rely heavily on the tools in order to support the reuse of requirements. All experiment based studies (S1, S5, S15, S17, S27, S54) barring one (S42) used the tool as support for reuse.

Generalization of results: The context and settings used in the experiments are not clearly described. The comparison of proposed approach with contemporary approaches is missing from all the studies present in Table 16. Different objectives of these studies and different contexts make the generalization of the results a cumbersome task.

4.3.5. Reflection on evaluated studies

Overall with regard to analysis and evaluation of these empirical studies we can conclude the following:

- The domain and context specific requirement reuse approaches make it difficult to generalize and use the results for other domains.
- Document examination is the most frequently used method for analysis of the requirements used by requirement reuse approaches.
- The degree of reuse is the most common measured variable. However, studies have a different understanding of this variable.
- The requirement reuse approaches can be integrated into tools as well as development processes. Many studies (S1, S5, S11, S15, S17, S19, S20, S22, S26, S27, S36, S41, S45, S54, S56) have demonstrated the effectiveness of reuse approaches with help of similar integration.

4.4. RQ3: what was the rigor and relevance of empirical studies on requirements reuse approaches?

The rigor and relevance evaluations were conducted on the studies that describe industrial evaluation. The data extraction was carried out for each attribute of rigor and relevance criteria, described in Section 3.4. One author conducted this evaluation and later another author reviewed the evaluation separately in order to reconfirm the evaluation. The evaluation results of the identified studies are presented in Tables 17 and 18.

The results show that the studies scored higher on the relevance as

Table 17
Results obtained from evaluation of case studies.

Study	Rigor rubrics				Relevance rubrics					Total score
	Context	Study design	Validity	Sum	Subject	Context	Scale	Research method	Sum	
S2	1	1	0	2	1	1	1	1	4	6
S8	0.5	1	0	1.5	1	1	1	1	4	5.5
S11	1	0.5	0	1.5	1	1	1	1	4	5.5
S19	1	0.5	0	1.5	1	1	1	1	4	5.5
S20	1	1	0	2	1	1	1	1	4	6
S22	1	1	1	3	1	1	1	1	4	7
S26	1	0.5	0	1.5	1	1	1	1	4	5.5
S34	1	0.5	0	1.5	1	1	1	1	4	5.5
S36	1	0.5	0	1.5	1	1	1	1	4	5.5
S38	0.5	0.5	0	1	1	1	1	1	4	5
S41	1	1	1	3	1	1	1	1	4	7
S45	1	0.5	0	1.5	1	1	1	1	4	5.5
S48	0.5	0	0	0.5	0	1	1	1	3	3.5
S56	1	1	0	2	1	1	1	1	4	6
S61	1	0.5	0	1.5	1	1	1	1	4	5.5

Table 18
Results obtained from evaluation of experiments.

Study	Rigor rubrics				Relevance rubrics					Total score
	Context	Study design	Validity	Sum	Subject	Context	Scale	Research method	Sum	
S1	0.5	1	0	1.5	0	0	0.5	0	0.5	2
S5	1	1	0.5	2.5	1	0	0.5	1	2.5	5
S15	0.5	0	0	0.5	1	0	0.5	0	1.5	2
S17	1	0.5	0	1.5	1	0	0.5	0	1.5	3
S27	0.5	0.5	0	1	0	0	0.5	0	0.5	1.5
S42	0.5	0.5	0	1	0	0	0.5	0	0.5	1.5
S54	0.5	0.5	0	1	0	0	0.5	0	0.5	1.5

compared to the rigor. Barring one study, all the studies scored the maximum points on the relevance criteria. However, only two studies scored the maximum points on the rigor criteria. The threats to the validity of the studies were absent in the majority of the studies.

Two studies (S22 [22] and S41 [41]) were identified that scored the highest on rigor and relevance criteria i.e., 7 points. These provided all details on how the study was conducted; the study design, subjects and the details of the validation were present in the studies. The majority of the evaluated studies fall between 5.5 and 6 points. That means these studies were short on answering one of the required aspects but discussed most of the required aspects mentioned in the rigor and relevance criteria. One study scored low on rigor [48].

Overall, with regard to the evaluation, we can conclude that:

- From the results, we can see that the reuse approaches consisting of text-based reusable requirements are preferred over non text-based reusable requirement with respect to industrial evaluation. One-third of the evaluated industrial studies, consisting of different reuse approaches, used the text-based reusable requirements.
- Only two studies described all aspects of rigor and relevance. Generally, the evaluated studies lacked one or two attributes required in the rigor and relevance rubrics.
- Only one security requirements reuse study was evaluated in the industry.
- Case studies scored higher on rigor and relevance as compared to experiment based studies.

4.4.1. Characteristics of studies with high rigor and relevance scores

The studies scoring highest on rigor and relevance (S22 & S41) rubrics used requirement structuring and feature modeling to support requirements reuse. Both were case studies and discussed the applicability of proposed approaches on software product lines. The studies describing the combination of structuring as reuse approach and textual requirements as reusable artefacts have scored higher on rigor and relevance. This was evident in both case study and experiment based studies. The highest scoring studies utilised the tool support for the application of requirement reuse approaches.

4.4.2. Characteristics of studies with low rigor and relevance scores

The lowest scoring studies (S42 and S54) used machine learning and task models for supporting requirements reuse. It was observed that these studies were focused on describing the actual approach rather than describing the evaluation, context and research method used for evaluation. In general, the experiment based studies scored the lowest in rigor as the information about study design and context was not described in detail. Another reason for the studies to score low was missing information about the research method and context in the relevance part.

5. Discussion

This section describes the lessons learned during the identification and analysis of the studies. Each section below discusses different outcomes of the study such as lack of the validation and impact of the choice of reusable requirements.

Requirements reuse phases: Woo et al. suggested that requirements reuse consists of three main activities retrieving, adapting, and consolidating [42]. The analysis of the studies showed that very few studies address these three aspects of requirements reuse. Mainly, studies have described one aspect in detail while failing to describe the impact on the other two activities. The “Structuring” approaches (see Table 9) describe the “adaptation” of the requirements to facilitate the reuse. However, the studies do not describe the retrieving and consolidation of the requirements in detail. Similarly, the “matching”

approaches describe the retrieving part of requirements reuse but do not describe the other two aspects (adaptation and consolidation).

Solution proposals for requirements reuse: The snowball sampling was conducted to identify the studies regardless of the type of research methodology used in the studies. It was found that majority of the studies on requirements reuse describe a solution proposal with the help of a hypothetical example. These solutions start by describing an approach to reuse the requirements and later apply the approach on an example. The examples used in the studies vary from basic project requirements (such as library management system) to more complex industrial examples (not real industrial system). These proposals provide a good way to describe an approach but it is difficult to evaluate the strength of the proposal with respect to industrial relevance.

Requirement reuse approaches: The investigation towards the identified studies showed that "Structuring" of requirements is the most used approach to facilitate the reuse of requirements. The rigor and relevance analysis shows that the "Structuring" was present in most of the evaluated industrial studies. Many identified studies (in Table 9) do not describe the industrial evaluation. These approaches were theoretically described but the authors have not shared results of the industrial validation. Therefore, considerable work is needed for requirements reuse approaches to be evaluated in the industry before comparing the pros and cons of these approaches. It is hard to compare the approaches and their effect on requirements reuse because of the lack of validation. Furthermore, few identified studies were very similar and from the same authors (e.g., S1 & S12, S68 & S69, S5 & S67, S16 & S19, S2 & S9 & S48), these studies were analyzed without considering their similarity in order to keep traceability and reproducibility of snowball sampling.

Choice of artefacts: A reusable artefact is a fundamental part on which the whole reuse approach is based on. The identified studies show a variety of reusable artefacts that are used by the researchers and practitioners. Similar to the requirements reuse approaches, most of these artefacts were not evaluated in the industry. Many studies did not discuss the reusable artefacts instead these studies focus on explaining the reuse approach. The text-based artefacts are commonly used and evaluated. The results show two lessons with respect to reuse artefacts: (i) only few artefact independent reuse approaches are present in literature, (ii) text-based artefacts are easier to validate in the industry as compare to other artefacts, possibly because of ease of use.

Security requirements: The reuse of security requirements was discussed frequently in the identified studies. These studies were not focused on broader requirements reuse and were limited to the reuse of security requirements. Out of the final set, five studies discussed the methods to reuse security requirements [6,25,27,36,52]. These studies used different methods and different types of artefacts, see Tables 9 and 10. Only one study [36] described the validation in the industry.

Missing validity threats: The rigor criteria checks that if a study has discussed validity threats or not. It was found that the majority of the evaluated industrial studies do not describe the validity threats. Only 2 studies [22,41] discussed all four types of the validity threats while majority of the studies failed to discuss these threats. This lack of discussion on validity threats in studies makes it difficult for the reader to analyze and interpret the results of the studies.

Increase in requirements reuse studies: The analysis of the number of publications by year highlights that more requirements reuse studies have been published in the last 10 years compared to the previous years. Since 2002, the number of published requirements reuse studies has been greater than during the previous fourteen years (1988–2001). A similar trend is seen in the publication of studies with details of the industrial evaluation. This is a positive sign and points to importance and interest in the field of requirements reuse.

Saved effort / resources: The main benefit of reuse is to save effort and resources that are spent on writing new artefact and avoid costs through reuse [97]. However, we have found only one primary study (S41, [41]) that discusses saved effort and resources with respect to the reuse of requirements. The majority of primary studies discuss details, execution, and results of requirement reuse approaches but these studies do not discuss effort saved or effort avoided through reuse. This is a research gap that should be addressed in future studies on reuse of requirements.

Developed for reuse vs. developed with reuse: During the analysis of requirements reuse approaches, we have faced difficulty in finding a common abstraction level that can be used to describe each requirement reuse approach. The problem originates from the type of solution that requirements reuse approaches attempt to provide. Few approaches focused on developing with reuse while other approaches focused on developing for reuse. Similar problems were also reported by a previous study on reuse of security requirements [64]. In future, a separate study should address this research gap and help in proposing common abstraction level of requirements reuse approaches and reusable artefacts.

Comparison with related work: Several studies on the reuse of requirements have been reported for nearly three decades now. However, only two studies have attempted to review the literature on reuse of software requirements [76,96]. The start set of the snowball sampling included a study containing systematic literature review of requirements reuse studies [96]. This study by Azambuja et al. analyzed three different characteristics of identified studies such as methodologies, kinds of solution and applications. A similar analysis was done in our study as well i.e., each identified study was evaluated for characteristics such as research method, tool support, repository support and product line support. However, we broaden our study to further analyze the requirements reuse with respect to reusable artefact and reuse approaches. In addition to all this, we have performed an evaluation of the quality of industry-evaluated studies.

The comparison of results between this study and the study by Mussbacher and Kienzle shows the difference in search strategy (search strings and limited to RE conferences) and the different classification of reuse approaches [76]. The strategy developed by Mussbacher and Kienzle accommodate any study that is classified by keyword "Reusability" in requirements engineering conferences [76]. Many studies included by Mussbacher and Kienzle did not match the inclusion/exclusion criteria developed for this study (present in Section 3.3.2). The other fundamental difference is present in the classification of the reuse approaches. The approaches were classified based on different nomenclature in both studies.

This study adds to the work of Mussbacher and Kienzle [76] and Azambuja et al. [96] (i) by re-evaluating the identified studies using clearly defined inclusion/exclusion criteria, (ii) by assessing the characteristics (such as artefact type, tool support) of the requirements reuse approaches and (iii) by providing quality evaluation of the studies through rigor and relevance criteria. The difference in primary studies highlights the importance of the search strategies and inclusion and exclusion criteria of literature reviews as pointed out by Kitchenham [84].

6. Conclusion

The reuse of requirements can help in the identification of possibilities of early reuse in product life cycle. Researchers have claimed that early reuse can lead to greater benefits in terms of reducing costs and development time as compared to reuse of code. The objective of this study was to identify the software requirement reuse approaches, identify important lessons from analysis of these identified approaches and perform a quality assessment of the identified empirical studies.

The snowball sampling technique helps in the identification of primary studies, similar to systematic literature review performed using database search. A start set of eleven articles was developed and each article in the start set was inspected for (i) reference list, and (ii) citations of articles using Google Scholar. Each article present in the reference list and citation list was analyzed for inclusion and exclusion

criteria. Similarly, five iterations were performed before we reached the state where no new article was identified. Sanity checks were performed using different reviewer and database search. In total 69 studies were identified that matched the inclusion and exclusion criteria present in the study. These studies were then investigated and a data extraction form was used to extract relevant data. To further evaluate the quality of empirical studies, rigor and relevance evaluation were performed. Following are the research questions addressed in this article and the respective outcome of these questions.

RQ1: What are the different software requirements reuse approaches present in literature? The analysis of sixty-nine primary studies, that were found using the snowball sampling, shows that eleven different requirements reuse approaches are present in the literature. Structuring of the requirements is the most commonly used approach for increasing the reusability of the artefact. Other identified approaches that are described in studies are matching similar requirements, analogy, parametrization of the requirements and combination of these approaches. These approaches are based on different types of reusable requirements e.g., textual form, abstract requirements, use cases etc.,

RQ2: How have the requirements reuse approaches identified in RQ1 been evaluated, and what was the outcome? It was found that only a subset of identified approaches have been evaluated in the industry. Twenty-two studies (15 case studies and 7 experiments) were evaluated with respect to reuse approach, data collection, measured variables and important lessons. It was found that text-based requirement reuse approaches are frequently evaluated in the industry. It was observed that empirical studies have only described the positive results with respect to requirement reuse, the negative impact of approaches are not discussed at all. Document examination is the most commonly used method for the analysis of requirements. The results of evaluated studies were difficult to generalize, as the context and validity threats are not discussed in many studies.

RQ3: What was the rigor and relevance of empirical studies on requirements reuse approaches? The rigor and relevance evaluation shows that the case study based articles have described the details better than the experiment based studies. The analysis of the rigor and relevance scores show the following important conclusions: (i) studies based on requirement structuring, as a reuse approach, or text, as the requirements form, have scored higher on rigor and relevance scales, (ii) the highest scoring studies have discussed the impact of proposed reuse approach in software product lines context, (iii) the supporting tools, implementing the requirements reuse approach, have no distinct impact on the rigor and relevance of studies, (iv) experiment based studies have scored lower in rigor and relevance since these studies lack detail of study design, context and research method used and, (v) the reuse approaches do not discuss the effort saved and avoided through reuse.

Nearly two-third of the requirements reuse approaches are not validated in the industry. The analysis of the non-validated studies does not show any reason behind this lack of evaluation. In future, authors would like to expand on this finding and try to identify the reasons behind this lack of industrial validation of requirements reuse approaches. Furthermore, varieties of different tools are used by different requirement reuse approaches and these tools can help in automating the requirement reuse steps. A separate study is required to analyze the working of these tools, their strengths, and weaknesses.

Acknowledgment

The research presented in this paper was partly funded by the Swedish Knowledge Foundation, and the project “Professional Licentiate of Engineering Research School”, conducted at Blekinge Institute of Technology, Software Engineering Research Lab - Sweden.

References

Primary Studies

[1][S1] S.C. Chou, J.Y. Chen, C.G. Chung, A behavior-based classification and retrieval technique for object-oriented specification reuse, Software-practice Experience 26 (7) (1996) 815–832.
[2][S2] W. Lam, A case-study of requirements reuse through product families, Ann. Softw. Eng. 5 (1) (1998) 253–277.
[3][S3] X. Franch, C. Palomares, C. Quer, S. Renault, F. De Lazzer, A metamodel for software requirement patterns, Requirements Engineering: Foundation for Software Quality, Springer Berlin Heidelberg, 2010, pp. 85–90.
[4][S4] A. Monzon, A practical approach to requirements reuse in product families of on-board systems, International Requirements Engineering, 2008, IEEE, 2008, pp. 223–228. RE'08. 16th IEEE.
[5][S5] C.L. Pacheco, I.A. Garcia, J.A. Calvo Manzano, M. Arcilla, A proposed model for reuse of software requirements in requirements catalog, J. Softw 27 (1) (2015) 1–21.
[6][S6] G. Sindre, D.G. Firesmith, A.L. Opdahl, A reuse-based approach to determining security requirements, Proceedings of the 9th International Workshop on Requirements Engineering: Foundation for Software Quality (REFSQ?03), (2003). Klagenfurt, Austria.
[7][S7] S. Paydar, M. Kahani, A semantic web enabled approach to reuse functional requirements models in web engineering, Autom. Softw. Eng. 22 (2) (2015) 241–288.
[8][S8] N. Niu, J. Savolainen, Z. Niu, M. Jin, J.R.C. Cheng, A systems approach to product line requirements reuse, Syst. J. IEEE 8 (3) (2014) 827–836.
[9][S9] W. Lam, Achieving requirements reuse: a domain-specific approach from avionics, J. Syst. Softw. 38 (3) (1997) 197–209.
[10][S10] K. Chen, W. Zhang, H. Zhao, H. Mei, An approach to constructing feature models based on requirements clustering, Requirements Engineering, 2005. Proceedings. 13th IEEE International Conference on, IEEE, 2005, pp. 31–40.
[11][S11] M. Moon, K. Yeom, H.S. Chae, An approach to developing domain requirements as a core asset based on commonality and variability analysis in a product line, Softw. Eng. IEEE Trans. 31 (7) (2005) 551–569.
[12][S12] S.C. Chou, J.Y. Chen, C.G. Chung, An executable specification language for specification understanding in object-oriented specification reuse, Inf. Softw. Technol. 38 (6) (1996) 419–434.
[13][S13] C. Montabert, D.S. McCrickard, W.W. Winchester, M.A. Prez-Quiones, An integrative approach to requirements analysis: How task models support requirements reuse in a user–centric design framework, Interacting with Comput. 21 (4) (2009) 304–315.
[14][S14] P. Massonet, A. Van Lamsweerde, Analogical reuse of requirements frameworks, Requirements Engineering, 1997., Proceedings of the Third IEEE International Symposium on, IEEE, 1997, pp. 26–37.
[15][S15] G. Spanoudakis, Analogical reuse of requirements specifications: A computational model, Appl. Artif. Intell. 10 (4) (1996) 281–305.
[16][S16] N. Maiden, Analogy as a paradigm for specification reuse, Softw. Eng. J. 6 (1) (1991) 3–16.
[17][S17] F.B.V. Benitti, R.C. da Silva, Evaluation of a systematic approach to requirements reuse, J. Universal Comput. Sci. 19 (2) (2013) 254–280.
[18][S18] C. Wild, D. Rosca, Evolution and reuse of formal specifications using decision structures, Knowledge-Based Software Engineering Conference, 1994. Proceedings., Ninth, IEEE, 1994, pp. 108–115.
[19][S19] N. Maiden, A. Sutcliffe, Exploiting reusable specifications through analogy, Commun. ACM 35 (4) (1992) 55–64.
[20][S20] D. Hauksdóttir, N.H. Mortensen, P.E. Nielsen, Identification of a reusable requirements structure for embedded products in a dynamic market environment, Comput. Indus. 64 (4) (2013) 351–362.
[21][S21] M.L. Griss, J. Favaro, M.D. Alessandro, Integrating feature modeling with the RSEB, Software Reuse, 1998. Proceedings. Fifth International Conference on, IEEE, 1998, pp. 76–85.
[22][S22] M. Eriksson, J. Börstler, K. Borg, Managing requirements specifications for product lines?an approach and industry case study, J. Syst. Softw. 82 (3) (2009) 435–447.
[23][S23] K. Ryan, B. Mathews, Matching conceptual graphs as an aid to requirements re-use, Requirements Engineering, 1993., Proceedings of IEEE International Symposium on, IEEE, 1993, pp. 112–120.
[24][S24] B. Moros, A. Toval, C. Vicente Chicote, Metamodeling variability to enable requirements reuse, Proceedings of EMMSAD (p. 141), (2008).
[25][S25] J.L. Velasco, R. Valencia Garcia, J.T. Fernandez-Breis, A. Toval, Modelling reusable security requirements based on an ontology framework, J. Res. Practice Inf. Technol. 41 (2) (2009). 119
[26][S26] E.K. Karatas, B. Iyidir, A. Birturk, Ontology-based software requirements reuse: Case study in fire control software product line domain, Data Mining Workshop (ICDMW), 2014 IEEE International Conference on, IEEE, 2014, pp. 832–839.
[27][S27] O. Daramola, G. Sindre, T. Stalhane, Pattern-based security requirements specification using ontologies and boilerplates, Requirements Patterns (RePa), 2012 IEEE Second International Workshop on, IEEE, 2012, pp. 54–59.
[28][S28] A. Finkelstein, Re-use of formatted requirements specifications, Softw. Eng. J. 3 (5) (1988) 186–197.
[29][S29] L. Melikhova, A. Elcock, A.A. Dovzhikov, G. Bulatov, D.O. Vavilov, Reengineering for system requirements reuse: methodology and use-case,

2007 IEEE International Symposium on Consumer Electronics, (2007).
[30][S30] J.L. Cybulski, K. Reed, Requirements classification and reuse: crossing domain boundaries, Software Reuse: Advances in Software Reusability, Springer Berlin Heidelberg, 2000, pp. 190–210.
[31][S31] D. Schmitz, H.W. Nissen, M. Jarke, T. Rose, P. Drews, F.J. Hesseler, M. Reke, Requirements engineering for control systems development in small and medium-sized enterprises, International Requirements Engineering, 2008. RE'08. 16th IEEE, IEEE, 2008, pp. 229–234.
[32][S32] S. Konrad, B.H. Cheng, Requirements patterns for embedded systems, Requirements Engineering, 2002. Proceedings. IEEE Joint International Conference on, IEEE, 2002, pp. 127–136.
[33][S33] M. Shehata, A. Eberlein, J. Hoover, Requirements reuse and feature interaction management, 15th International Conference on Software & Systems Engineering and their Applications (ICSSEA?02), (2002). Paris, France
[34][S34] D. Hauksdóttir, A. Vermehren, J. Savolainen, Requirements reuse at danfoss, 2012 20th IEEE International Requirements Engineering Conference, RE 2012 - Proceedings, IEEE, 2012, pp. 309–314. 10.1109/RE.2012.6345820
[35][S35] E. Perednikas, Requirements reuse based on forecast of user needs, Proceedings of the 20th EURO Mini Conference on Continuous Optimization and Knowledge-Based Technologies, (2008), pp. 450–455. Neringa, Lithuania
[36][S36] A. Toval, J. Nicolas, B. Moros, F. Garcia, Requirements reuse for improving information systems security: a practitioner?s approach, Requir. Eng. 6 (4) (2002) 205–219.
[37][S37] H. Gomaa, Reusable software requirements and architectures for families of systems, J. Syst. Softw. 28 (3) (1995) 189–202.
[38][S38] S. Lee, H. Ko, M. Han, D. Jo, J. Jeong, K. Kim, Reusable sw requirements development process: Embedded sw industry experiences, Software Engineering Conference, 2007. ASWEC 2007. 18th Australian, IEEE, 2007, pp. 147–158.
[39][S39] R. Gotzhein, M. Kronenburg, C. Peper, Reuse in requirements engineering: Discovery and application of a real-time requirement pattern, Formal Techniques in Real-Time and Fault-Tolerant Systems, Springer Berlin Heidelberg, 1998, pp. 65–74.
[40][S40] S. Castano, V. De Antonellis, Reuse of conceptual requirement specifications, Requirements Engineering, 1993., Proceedings of IEEE International Symposium on, IEEE, 1993, pp. 121–124.
[41][S41] L. Goldin, D.M. Berry, Reuse of requirements reduced time to market at one industrial shop: a case study, Requir. Eng. 20 (1) (2015) 23–44.
[42][S42] H.G. Woo, W.N. Robinson, Reuse of scenario specifications using an automated relational learner: a lightweight approach. in requirements engineering, 2002 Proceedings. IEEE Joint International Conference on, IEEE, 2002, pp. 173–180.
[43][S43] B. Bonilla-Morales, S. Crespo, C. Clunie, Reuse of use cases diagrams: an approach based on ontologies and semantic web technologies, Int. J. Comput. Sci 9 (1) (2012) 24–29.
[44][S44] R. Pooley, C. Warren, Reuse through requirements traceability, Software Engineering Advances, 2008. ICSEA'08. The Third International Conference on, IEEE, 2008, pp. 65–70.
[45][S45] M. Mannion, H. Kaindl, J. Wheadon, B. Keepence, Reusing single system requirements from application family requirements, Proceedings of the 21st international conference on Software engineering, ACM, 1999, pp. 453–462.
[46][S46] M.C. Blok, J.L. Cybulski, Reusing UML specifications in a constrained application domain, Software Engineering Conference, 1998. Proceedings.1998 Asia Pacific, IEEE, 1998, pp. 196–202.
[47][S47] M. Saeki, Reusing use case descriptions for requirements specification: towards use case patterns, Software Engineering Conference, 1999.(APSEC'99) Proceedings. Sixth Asia Pacific, IEEE, 1999, pp. 309–316.
[48][S48] W. Lam, Scenario reuse: A technique for complementing scenario-based requirements engineering approaches, Software Engineering Conference, 1997. Asia Pacific... and International Computer Science Conference 1997. APSEC'97 and ICSC'97. Proceedings, IEEE, 1997, pp. 332–341.
[49][S49] W.L. Johnson, D.R. Harris, Sharing and reuse of requirements knowledge, Knowledge-Based Software Engineering Conference, 1991. Proceedings., 6th Annual, IEEE, 1991, pp. 57–66.
[50][S50] B. Keepence, M. Mannion, S. Smith, SMARTRe requirements: writing reusable requirements, Systems Engineering of Computer Based Systems, 1995., Proceedings of the 1995 International Symposium and Workshop on, IEEE, 1995, pp. 27–34.
[51][S51] K. Periyasamy, J. Chidambaram, Software reuse using formal specification of requirements, Proceedings of the 1996 conference of the Centre for Advanced Studies on Collaborative research (p. 31), IBM Press, 1996.
[52][S52] D.G. Firesmith, Analysing and specifying reusable security requirements, J. Object Technol. 3 (1) (2003).
[53][S53] A. Sutcliffe, N. Maiden, Supporting component matching for software reuse, Advanced Information Systems Engineering, Springer Berlin Heidelberg, 1992, pp. 290–303.
[54][S54] C. Montabert, D. Bussert, S.S. Gifford, C.M. Chewar, D.S. Mc Crickard, Supporting requirements reuse in notification systems design through task modeling, Proc. 11th International Conference on Human-Computer Interaction (HCII ?05), (2005).
[55][S55] A.G. Sutcliffe, N.A. Maiden, S. Minocha, D. Manuel, Supporting scenario-based requirements engineering, Softw. Eng. IEEE Trans. 24 (12) (1998) 1072–1088.
[56][S56] R. Biddle, J. Noble, E. Tempero, Supporting reusable use cases, Software Reuse: Methods, Techniques, and Tools, Springer Berlin Heidelberg, 2002, pp. 210–226.
[57][S57] A. Von Knethen, B. Paech, F. Kiedaisch, F. Houdek, Systematic requirements recycling through abstraction and traceability, Requirements Engineering, 2002. Proceedings. IEEE Joint International Conference on, IEEE, 2002, pp. 273–281.
[58][S58] J. Naish, L. Zhao, Towards a generalised framework for classifying and retrieving requirements patterns, Requirements Patterns (RePa), 2011 First International Workshop on, IEEE, 2011, pp. 42–51.
[59][S59] I. Alexander, F. Kiedaisch, Towards recyclable system requirements, Engineering of Computer-Based Systems, 2002. Proceedings. Ninth Annual IEEE International Conference and Workshop on the, IEEE, 2002, pp. 9–16.
[60][S60] N. Heumesser, F. Houdek, Towards systematic recycling of systems requirements, Software Engineering, 2003. Proceedings. 25th International Conference on, IEEE, 2003, pp. 512–519.
[61][S61] M. Mannion, H. Kaindl, Using parameters and discriminants for product line requirements, Syst. Eng. 11 (1) (2008) 61–80.
[62][S62] C. Schmitt, P. Liggesmeyer, A model for structuring and reusing security requirements sources and security requirements, REFSQ Workshops, Vancouver, 2015, pp. 34–43.
[63][S63] J. Dehlinger, R.R. Lutz, A product-line requirements approach to safe reuse in multi–agent systems, ACM SIGSOFT Software Engineering Notes (Vol. 30, No. 4, ACM, 2005, pp. 1–7.
[64][S64] M. Marrero, S. SanchezCuadrado, A. Fraga, J. Llorens, Applying ontologies and intelligent text processing in requirements reuse. in first workshop on knowledge reuse (KREUSE?08), 2008, 25-29.
[65][S65] C.C. Chiang, D. Neubart, Constructing reusable specifications through analogy, Proceedings of the 1999 ACM symposium on Applied computing, ACM, 1999, pp. 586–592.
[66][S66] O. Lopez, M.A. Laguna, F.J. Garcia, Reuse based analysis and clustering of requirements diagrams, Pre-Proceedings of the Eighth International Workshop on Requirements Engineering: Foundation for Software Quality (REFSQ?02), (2002), pp. 71–82.
[67][S67] C. Pacheco, I. Garcia, J.A. . Calvo-Manzano, M. Arcilla, Reusing functional software requirements in small-sized software enterprises: a model oriented to the catalog of requirements, Requir. Eng. (2016) 1–13. Vancouver
[68][S68] N.H. Bakar, Z.M. Kasirun, N. Salleh, Terms extractions: An approach for requirements reuse, In Information Science and Security (ICISS), 2015 2nd International Conference on, IEEE, 2015, pp. 1–4.
[69][S69] N.H. Bakar, Z.M. Kasirun, H.A. Jalab, Towards requirements reuse: Identifying similar requirements with latent semantic analysis and clustering algorithms, Proc. of the Second Intl. Conf. on Advances In Computing, Communication and Information Technology-CCIT 2014, (2014), pp. 19–24.

Other References

[70] W.K. Charles, Software reuse, ACM Comput. Surv. 24 (2) (1992) 131–183, http://dx.doi.org/10.1145/130844.130856.
[71] W. Frakes, C. Terry, Software reuse: metrics and models, ACM Comput. Surv. (CSUR) 28 (2) (1996) 415–435.
[72] T. Will, Where does reuse start?. SIGSOFT softw. eng. notes 15, 2 1990, 1990, 42–46, 10.1145/382296.382702.
[73] T. Ravichandran, M.A. Rothenberger, Software reuse strategies and component markets, Commun. ACM 46 (8) (2003) 109–114.
[74] P. Mohagheghi, R. Conradi, Quality, productivity and economic benefits of software reuse: a review of industrial studies. Empirical Software Engineering 12 (5) (2007) 471–516.
[75] V. Karakostas., 1989. requirements for CASE tools in early software reuse, SIGSOFT Softw. Eng. Notes 14 2 (1989) 39–41, http://dx.doi.org/10.1145/71647.71650.
[76] G. Mussbacher, J. Kienzle, A vision for generic concern-oriented requirements reuse re@ 21, Requirements Engineering Conference (RE), 2013 21st IEEE International, IEEE, 2013, pp. 238–249.
[77] J. Radatz, A. Geraci, F. Katki, IEEE standard glossary of software engineering terminology, IEEE Std (1990) 3. 610121990(121990)
[78] S. Buhne, K. Lauenroth, K. Pohl, Modelling requirements variability across product lines, 13th IEEE International Conference on Requirements Engineering (RE'05), IEEE, 2005, pp. 41–50.
[79] M. Ivarsson, T. Gorschek, A method for evaluating rigor and industrial relevance of technology evaluations, Empirical Softw. Eng. 16 (3) (2011) 365–395.
[80] D.I. Sjoberg, B. Anda, E. Arisholm, T. Dyba, M. Jorgensen, A. Karahasanovic, M. Vokác, Conducting realistic experiments in software engineering, Empirical Software Engineering, 2002. Proceedings. 2002 International Symposium n, IEEE, Chicago, 2002, pp. 17–26.
[81] T. Dyba, B.A. Kitchenham, M. Jorgensen, Evidence-based software engineering for practitioners. software, IEEE 22 (1) (2005) 58–65.
[82] C. Wohlin, An analysis of the most cited articles in software engineering journals-2000, Inf. Softw. Technol. 49 (1) (2007) 2–11. Chicago
[83] W. Lam, S. Jones, C. Britton, Technology transfer for reuse: a management model and process improvement framework. requirements engineering, Proceedings. 1998 Third International Conference on, IEEE, 1998, pp. 233–240.
[84] B. Kitchenham, Procedures for performing systematic reviews. keele, 2004, UK, Keele University, 33. 1–26.
[85] C. Wohlin, Guidelines for snowballing in systematic literature studies and a replication in software engineering, Proceedings of the 18th International Conference on Evaluation and Assessment in Software Engineering, ACM, 2014, p. 38.
[86] D. Badampudi, C. Wohlin, K. Petersen, Experiences from using snowballing and

database searches in systematic literature studies, Proceedings of the 19th International Conference on Evaluation and Assessment in Software Engineering, ACM, 2015, p. 17.
[87] B.A. Kitchenham, S.L. Pfleeger, L.M. Pickard, P.W. Jones, D.C. Hoaglin, K. ElEmam, J. Rosenberg, Preliminary guidelines for empirical research in software engineering, in software engineering, IEEE Trans. 28 (8) (2002) 721–734, http://dx.doi.org/10.1109/TSE.2002.1027796.
[88] A. Jedlitschka, M. Ciolkowski, Towards evidence in software engineering. in empirical software engineering, ISESE'04. Proceedings. 2004 International Symposium on, IEEE, 2004, pp. 261–270.
[89] F. Elberzhager, A. Rosbach, J. MuNch, R. Eschbach, Reducing test effort: A systematic mapping study on existing approaches, Inf. Softw. Technol. 54 (10) (2012) 1092–1106.
[90] T. Dybå, T. Dingsø yr, Strength of evidence in systematic reviews in software engineering, Proceedings of the Second ACM-IEEE international symposium on Empirical software engineering and measurement, ACM, 2008, pp. 178–187.
[91] H. Munir, M. Moayyed, K. Petersen, Considering rigor and relevance when evaluating test driven development: A systematic review, Inf. Softw. Technol. 56 (4) (2014) 375–394.
[92] N.B. Ali, K. Petersen, C. Wohlin, A systematic literature review on the industrial use of software process simulation, J. Syst. Softw. 97 (2014) 65–85.
[93] H. Munir, K. Wnuk, P. Runeson, Open innovation in software engineering: a systematic mapping study, Empirical Softw. Eng. (2015) 1-40.
[94] W. Lam, J.A. McDermid, A.J. Vickers, Ten steps towards systematic requirements reuse, Requir. Eng. 2 (2) (1997) 102113, http://dx.doi.org/10.1007/BF02813029.
[95] D. Hauksdóttir, N.H. Mortensen, P.E. Nielsen, Identified adjustability dimensions when generating a product specific requirements specification by requirements reuse, Comput. Indus. 65 (6) (2014) 952–966.
[96] F.B. de Azambuja, R.M. Bastos, A.P.T. Bacelo, Systematic review of requirements reuse. in SEKE, 2009, 67-72.
[97] M. Irshad, R. Torkar, K. Petersen, W. Afzal, Capturing cost avoidance through reuse: systematic literature review and industrial evaluation, Proceedings of the 20th International Conference on Evaluation and Assessment in Software Engineering, ACM, 2016, p. 35.
[98] B. Kitchenham, O.P. Brereton, D. Budgen, M. Turner, J. Bailey, S. Linkman, Systematic literature reviews in software engineering?a systematic literature review, Inf. Softw. Technol. 51 (1) (2009) 7–15.
[99] M. Bano, D. Zowghi, N. Ikram, Systematic reviews in requirements engineering: A tertiary study, Empirical Requirements Engineering (empire), 2014 Ieee Fourth International Workshop on, IEEE, 2014, pp. 9–16.
[100] M.I. Babar, M. Ghazali, D.N. Jawawi, Systematic reviews in requirements engineering: A systematic review. In Software Engineering Conference (MySEC), 2014 8th Malaysian, IEEE, 2014. 43-48
[101] R.P.D. Redondo, J.J.P. Arias, A.F. Vilas, B.B. Martnez, Approximate retrieval of incomplete and formal specifications applied to vertical reuse, Software Maintenance, 2002. Proceedings. International Conference on, IEEE, 2002, pp. 618–627.
[102] A. Fuggetta, A classification of CASE technology, in Computer 26(12) (1993) 25–38.
[103] H.W. Nissen, M. Jarke, Repository support for multi-perspective requirements engineering, Inf. Syst. 24 (2) (1999) 131–158.
[104] S. Apel, C. Kästner, An overview of feature-oriented software development, J. Object Technol. 8 (5) (2009) 49–84.
[105] I. Jacobson, P.W. Ng, Aspect-oriented software development with use cases (addison-wesley object technology series). addison-wesley professional, 2004.
[106] B. Kitchenham, What's up with software metrics. a preliminary mapping study, J. Syst. Softw. 83 (1) (2010) 37–51.
[107] D.S. Cruzes, T. Dybå, Research synthesis in software engineering: A tertiary study, Inf. Softw. Technol. 53 (5) (2011) 440–455. Chicago
[108] K. Petersen, C. Gencel, Worldviews, research methods, and their relationship to validity in empirical software engineering research, Software Measurement and the 2013 Eighth International Conference on Software Process and Product Measurement (IWSM-MENSURA), 2013 Joint Conference of the 23rd International Workshop on, IEEE, 2013, pp. 81–89.
[109] R.A. Silva, S.D.R.S. de Souza, P.S.L. de Souza, A systematic review on search based mutation testing. information and software technology., 2016.
[110] K. Wnuk, R.K. Kollu, A systematic mapping study on requirements scoping, Proceedings of the 20th International Conference on Evaluation and Assessment in Software Engineering, ACM, 2016, p. 32.
[111] "scopus." scopus - welcome to scopus. n.p., n.d. web. 19 apr, 2017, https://www.scopus.com/.
[112] Data collection sheet requirements reuse, 2017, Retrieved from https://goo.gl/KgZihP.